

## Unit 1. Concepts of OOPS

1.1 Difference between procedural programming and OOPS

1.2 Various library(header) files require for C++

1.3 Data types in C++

1.4 Concepts of String

1.4.1 character Array

1.4.2 pointer to character array

1.4.3 Use of String.h and its important functions  
(strcmp, strcat, strcpy, strlen, strrev )

1.5 Concepts of Class and Objects

### 1.1 Difference between procedural programming and OOPS

**OOP** refers to Object-Oriented Programming, and it deals with objects and their properties. The major concepts of OOP are classes/objects, abstraction, encapsulation, polymorphism, and inheritance.

**POP** refers to Procedural-Oriented Programming and deals with programs and functions. In POP, programs are divided into functions, and data is global.

Following are the important differences between OOP and POP.

| No. | Key                   | OOP                                               | POP                                                  |
|-----|-----------------------|---------------------------------------------------|------------------------------------------------------|
| 1   | Definition            | OOP stands for Object Oriented Programming.       | POP stands for Procedural Oriented Programming.      |
| 2   | Approach              | OOP follows a bottom up approach.                 | POP follows a top down approach.                     |
| 3   | Division              | A program is divided into objects.                | A program is divided into functions.                 |
| 4   | Inheritance supported | Inheritance is supported.                         | Inheritance is not supported.                        |
| 5   | Access control        | Access control is supported via access modifiers. | No access modifiers are supported.                   |
| 6   | Data Hiding           | Encapsulation is used to hide data.               | No data hiding present. Data is globally accessible. |
| 7   | Example               | C++, Java                                         | C, Pascal                                            |

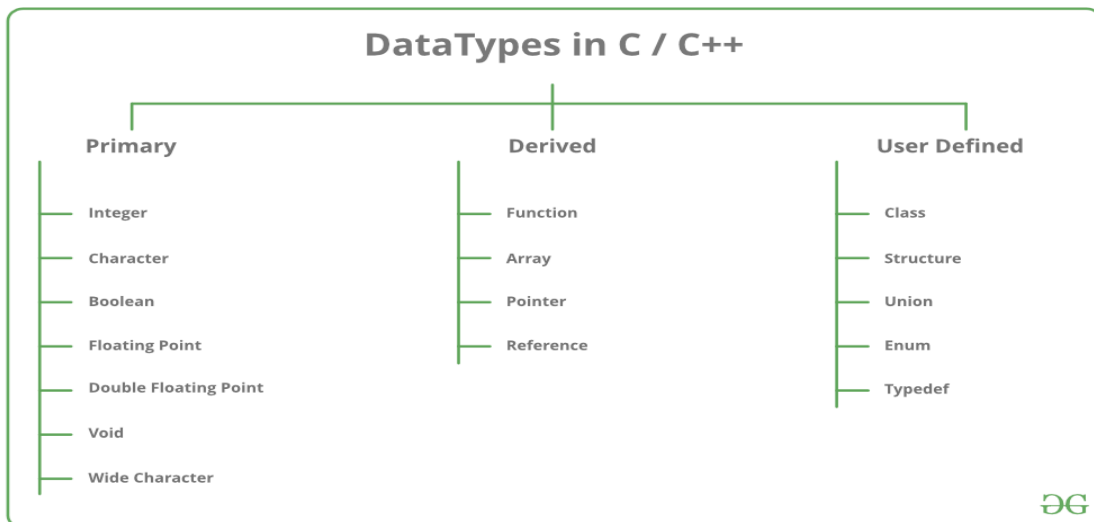
## 1.2 Various library(header) files require for C++

Standard Header Files And Their Uses:

1. `#include<stdio.h>`: It is used to perform input and output operations using functions `scanf()` and `printf()`.
2. `#include<iostream.h>`: It is used as a stream of Input and Output using `cin` and `cout`.
3. `#include<string.h>`: It is used to perform various functionalities related to string manipulation like `strlen()`, `strcmp()`, `strcpy()`, `size()`, etc.
4. `#include<math.h>`: It is used to perform mathematical operations like `sqrt()`, `log2()`, `pow()`, etc.
5. `#include<iomanip.h>`: It is used to access `set()` and `setprecision()` function to limit the decimal places in variables.
6. `#include<signal.h>`: It is used to perform signal handling functions like `signal()` and `raise()`.
7. `#include<stdarg.h>`: It is used to perform standard argument functions like `va_start()` and `va_arg()`. It is also used to indicate the start of the variable-length argument list and to fetch the arguments from the variable-length argument list in the program respectively.
8. `#include<errno.h>`: It is used to perform error handling operations like `errno()`, `strerror()`, `perror()`, etc.
9. `#include<fstream.h>`: It is used to control the data to read from a file as an input and data to write into the file as an output.
10. `#include<time.h>`: It is used to perform functions related to `date()` and `time()` like `setdate()` and `getdate()`. It is also used to modify the system date and get the CPU time respectively.
11. `#include<float.h>`: It contains a set of various platform-dependent constants related to floating point values. These constants are proposed by ANSI C. They allow making programs more portable. Some examples of constants included in this header file are `e(exponent)`, `b(base/radix)`, etc.
12. `#include<limits.h>`: It determines various properties of the various variable types. The macros defined in this header, limits the values of various variable types like `char`, `int`, and `long`. These limits specify that a variable cannot store any value beyond these limits, for example an unsigned character can store up to a maximum value of 255.
13. `#include<assert.h>`: It contains information for adding diagnostics that aid program debugging.
14. `#include<ctype.h>`: It contains function prototypes for functions that test characters for certain properties, and also function prototypes for functions that can be used to convert uppercase letters to lowercase letters and vice versa.
15. `#include<locale.h>`: It contains function prototypes and other information that enables a program to be modified for the current locale on which it's running. It enables the computer system to handle different conventions for expressing data such as times, dates or large numbers throughout the world.
16. `#include<setjmp.h>`: It contains function prototypes for functions that allow bypassing of the usual function call and return sequence.
17. `#include<stddef.h>`: It contains common type definitions used by C for performing calculations.

## 1.3 Data types in C++

Data types are used to specify the type of data a variable can store. The following are the basic data types available in C++.



Data types in C++ are mainly divided into three categories:

**Primitive Data Types:** These data types are built-in or predefined and can be used directly by the user to declare variables. Examples include int, char, float, bool, etc. The primitive data types available in C++ are:

Integer  
Character  
Boolean  
Floating Point  
Double Floating Point  
Valueless or Void  
Wide Character

**Derived Data Types:** These data types are derived from the primitive or built-in data types. The four main types are:

Function  
Array  
Pointer  
Reference

**Abstract or User-Defined Data Types:** These data types are defined by the user. For example, defining a class or a structure in C++. C++ provides the following user-defined data types:

Class  
Structure  
Union  
Enumeration  
Typedef

### Primitive data types available in C++

**Integer:** The keyword used for integer data types is int. Integers typically require 2 bytes of memory space and range from -2147483648 to 2147483647.

**Character:** The character data type is used for storing characters. The keyword used for the character

data type is char. Characters typically require 1 byte of memory space and range from -128 to 127 or 0 to 255.

**Boolean:** The boolean data type is used for storing boolean or logical values. A boolean variable can store either true or false. The keyword used for the boolean data type is bool.

**Floating Point:** The floating point data type is used for storing single-precision floating point values or decimal values. The keyword used for the floating point data type is float. Float variables typically require 4 bytes of memory space.

**Double Floating Point:** The double floating point data type is used for storing double-precision floating point values or decimal values. The keyword used for the double floating point data type is double. Double variables typically require 8 bytes of memory space.

**Void:** Void means "without any value." The void data type represents a valueless entity and is used for functions that do not return a value.

**Wide Character:** The wide character data type is a character data type with a size greater than the normal 8-bit data type. It is represented by wchar\_t and is generally 2 or 4 bytes long.

## 1.4 Concepts of String

### 1.4.1 Character Array

A string is actually a one-dimensional array of characters terminated by a null character '\0'. The following declaration and initialization create a string consisting of the word "Hello." To hold the null character at the end of the array, the size of the character array containing the string is one more than the number of characters in the word "Hello".

```
char greeting[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

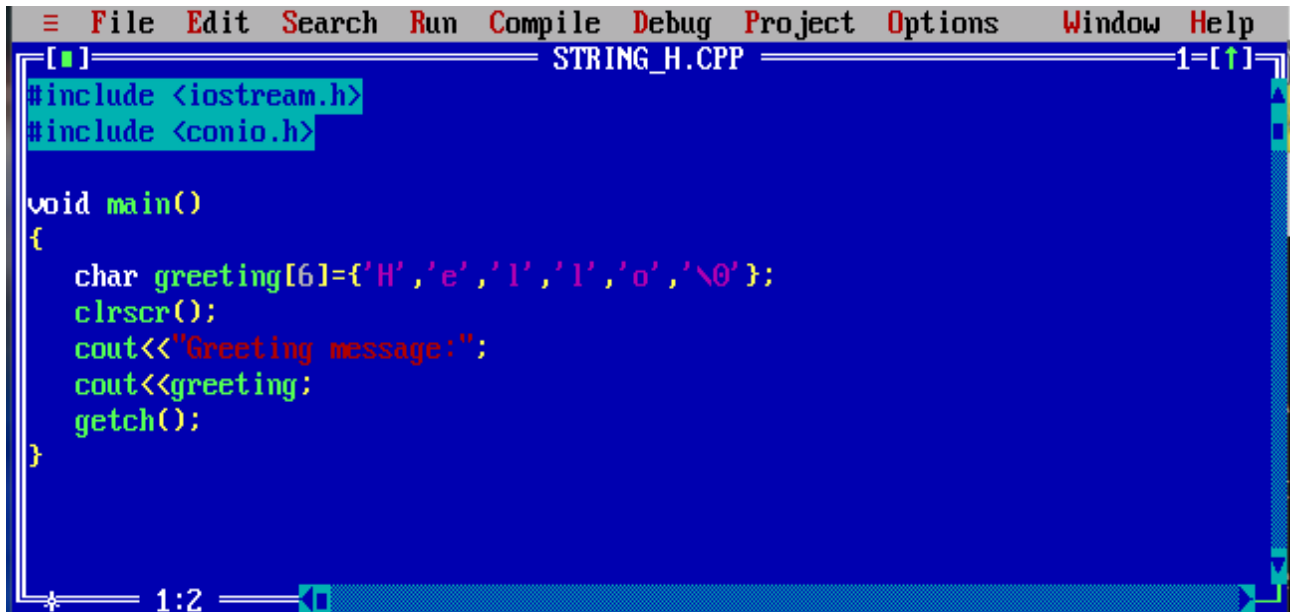
If you follow the rule of array initialization, you can write the above statement as follows.

```
char greeting[] = "Hello";
```

The following is the memory presentation of the above defined string in C/C++ .

|          |         |         |         |         |         |         |
|----------|---------|---------|---------|---------|---------|---------|
| Index    | 0       | 1       | 2       | 3       | 4       | 5       |
| Variable | H       | e       | l       | l       | o       | \0      |
| Address  | 0x23451 | 0x23452 | 0x23453 | 0x23454 | 0x23455 | 0x23456 |

Actually, you do not need to place the null character at the end of a string constant. The C++ compiler automatically places the '\0' at the end of the string when it initializes the array. Let us try to print the above-mentioned string.

**Example 1.1 String Initialization and Display.**


```

File Edit Search Run Compile Debug Project Options Window Help
STRING_H.CPP 1=[↑]
#include <iostream.h>
#include <conio.h>

void main()
{
    char greeting[6]={'H','e','l','l','o','\0'};
    clrscr();
    cout<<"Greeting message:";
    cout<<greeting;
    getch();
}
1:2

```

When the above code is compiled and executed, it produces the following result



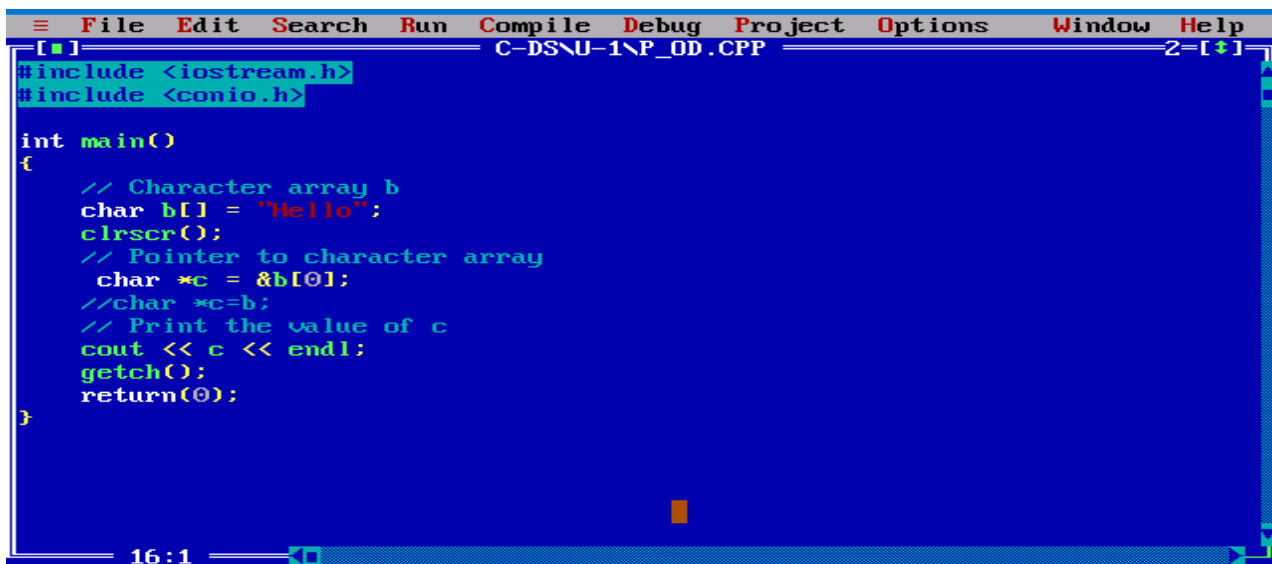
```

Greeting message:Hello_

```

**1.4.2 pointer to character array**

A pointer variable of the char type is treated as a string. The pointer can be assigned a character array. When the character pointer is assigned, **cout** prints the complete array of characters (till it encounters a null character).

**Example: Pointer to character array**


```

File Edit Search Run Compile Debug Project Options Window Help
C-DS\U-1\P_OD.CPP 2=[↑]
#include <iostream.h>
#include <conio.h>

int main()
{
    // Character array b
    char b[] = "Hello";
    clrscr();
    // Pointer to character array
    char *c = &b[0];
    //char *c=b;
    // Print the value of c
    cout << c << endl;
    getch();
    return(0);
}
16:1

```

This creates a string and stores its address in the pointer variable c. The pointer c now points to the first character of the string "Hello." The output of the above code is Hello.

```
Hello
```

### Example 1.2: Read and Display String Using Pointer.

```

File Edit Search Run Compile Debug Project Options Window Help
POINTER.CPP 1=[↑]
#include <iostream.h>
#include <conio.h>

void main()
{
    char *s;
    clrscr();
    cout<<"\n Enter string ";
    cin>>s;
    cout<<"\n String is "<<s;
    getch();
}
1:1

```

The above program takes a string from the user and displays the string in the output.

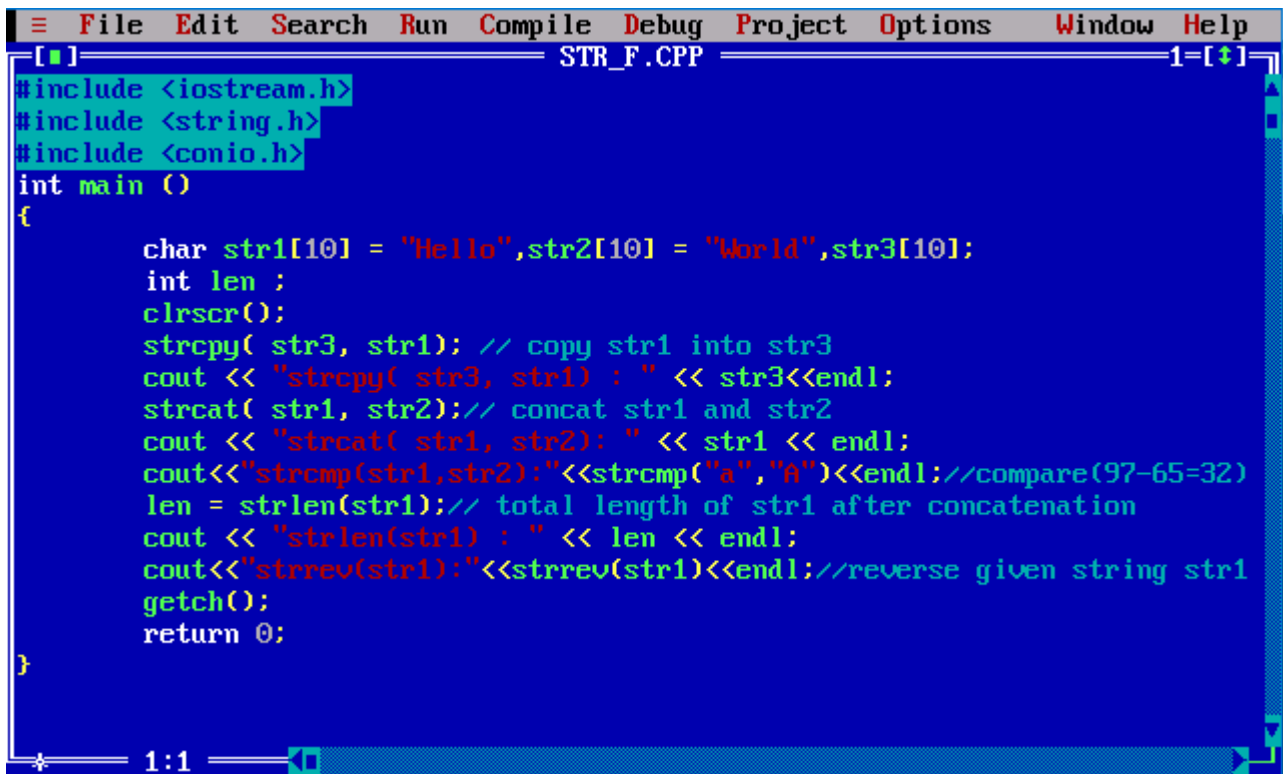
```
Enter string Hello
String is Hello
```

### 1.4.3 Use of String.h and its important functions (strcmp, strcat, strcpy, strlen, strrev )

| No | Function & Purpose                                                                                        |
|----|-----------------------------------------------------------------------------------------------------------|
| 1  | <b>strcpy(s1, s2)</b> copies the string s2 into the string s1.                                            |
| 2  | <b>strcat(s1, s2)</b> concatenates string s2 onto the end of string s1.                                   |
| 3  | <b>strlen(s1)</b> returns the length of the string s1.                                                    |
| 4  | <b>strcmp(s1, s2)</b> returns 0 if s1 and s2 are the same; less than 0 if s1<s2; greater than 0 if s1>s2. |
| 5  | <b>strrev(s1)</b> returns the reverse of the string s1.                                                   |

The following example makes use of the above-mentioned functions.

### Example 1.3 String functions.



```

File Edit Search Run Compile Debug Project Options Window Help
STR_F.CPP
#include <iostream.h>
#include <string.h>
#include <conio.h>
int main ()
{
    char str1[10] = "Hello", str2[10] = "World", str3[10];
    int len ;
    clrscr();
    strcpy( str3, str1); // copy str1 into str3
    cout << "strcpy( str3, str1) : " << str3<<endl;
    strcat( str1, str2); // concat str1 and str2
    cout << "strcat( str1, str2): " << str1 << endl;
    cout<<"strcmp(str1,str2):"<<strcmp("a","a")<<endl; //compare(97-65=32)
    len = strlen(str1); // total length of str1 after concatenation
    cout << "strlen(str1) : " << len << endl;
    cout<<"strrev(str1):"<<strrev(str1)<<endl; //reverse given string str1
    getch();
    return 0;
}
1:1

```

After compilation and execution, the program output will be as follows.

```

strcpy( str3, str1) : Hello
strcat( str1, str2): HelloWorld
strcmp(str1,str2):32
strlen(str1) : 10
strrev(str1):dlrowolleH

```

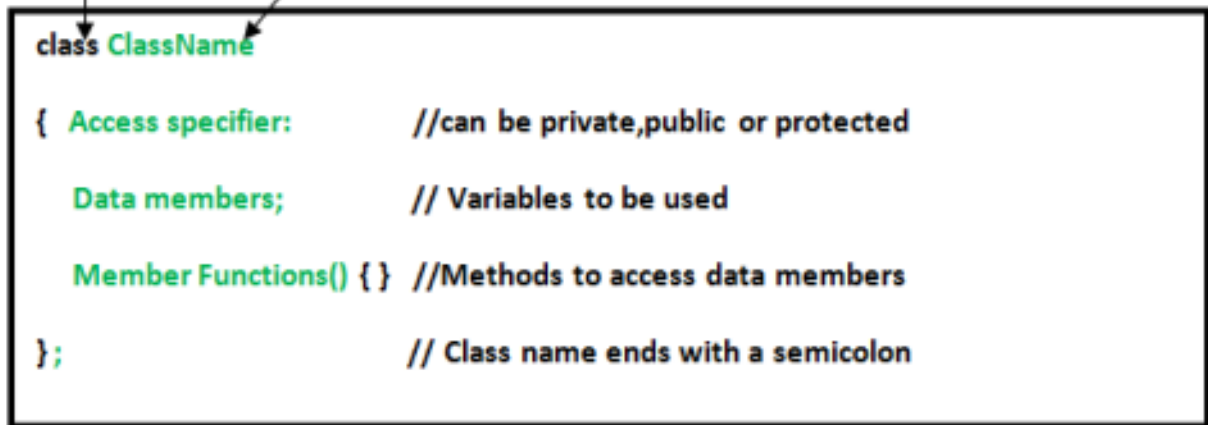
## 1.5 Concepts of Class and Objects

A class is a user-defined data type. It can be used like any other built-in data type and has data members and member functions. Data members are the data variables, and member functions are the functions used to manipulate these variables. Together, these data members and member functions are called class members.

### Defining Class and Declaring Objects

A class is defined in C++ using the keyword `class` followed by the name of the class. The body of class is defined inside the curly brackets and terminated by a semicolon at the end.

keyword      user-defined name



**Declaring Objects:** To use the data and access functions defined in the class, you need to create objects. Syntax: `ClassName ObjectName;`

**Accessing Data Members and Member Functions:** The data members and member functions of a class can be accessed using the dot('.') operator with the object.

Public data members are accessed in the same way as described above; however, private data members are not allowed to be accessed directly by the object. Accessing a data member depends on the access control of that data member. This access control is determined by access modifiers in C++. There are three access modifiers: public, private, and protected.

### Member Functions in Classes

There are two ways to define a member function:

1. Inside the class definition.
2. Outside the class definition.

To define a member function outside the class definition, we have to use the scope resolution :: operator along with the class name and function name.

#### Example 1.4: Write a program to find the area of a rectangle using a class.

```

#include <iostream.h>
#include <conio.h>

class rect
{
    private:
        int l,b;
    public:
        void getdata();
        void putdata()//inside class
        {
            cout<<"length "<<l<<endl;
            cout<<"breath "<<b<<endl;
            cout<<"area is "<<l*b<<endl;
        }
}
  
```



```

    }
};

void rect::getdata()//outside class
{
    cout<<"enter length";
    cin>>l;
    cout<<"enter breath";
    cin>>b;
}
void main()
{
    clrscr();
    rect r;
    r.getdata();
    r.putdata();
    getch();
}

```

**Output**

```

enter length10
enter breath10
length 10
breath 10
area is 100

```

**Programming exercises and solutions**

1. Write a program to input string and convert uppercase characters to lowercase.

```

#include <iostream.h>
#include <conio.h>

void main()
{
    int i;
    char s[10];
    clrscr();
    cout<<"Enter String:";
    cin>>s;
    cout<<"String is"<<s<<endl;

    for(i=0;s[i]!='\0';i++)
    {
        if(s[i]>=65 && s[i]<=90)
            s[i]=s[i]+32;
        cout<<s[i];
    }
    getch();
}

```

2. Write a program that takes multiple strings as input. Display each string along with its vowels, and finally, display the total number of vowels found in all strings.

```
#include <iostream.h>
#include <conio.h>

void main()
{
    int c=0;
    int total=0;
    int i,j,n;
    char s[6][10];
    clrscr();
    cout<<"HOW MANY STRING DO YOU WANT";
    cin>>n;
    cout<<"ENTER STRING";
    for(i=0;i<n;i++)
        cin>>s[i];

    for(i=0;i<n;i++)
        cout<<"\t\t"<<s[i]<<endl;

    for(i=0;i<n;i++)
    {
        cout<<"\t\t";
        for(j=0;s[i][j]!='\0';j++)
        {
            if(s[i][j]=='a' || s[i][j]=='e' || s[i][j]=='i' || s[i][j]=='u' || s[i][j]=='o')
                c++;
            cout<<s[i][j];
        }
        cout<<"\t"<<c;
        total+=c;
        c=0;
        cout<<endl;
    }
    cout<<"\t\t"<<"total vowels "<<total;
    getch();
}
```

3. Print the values of a char array using a pointer. Reverse the char array using a pointer.

```
#include <iostream.h>
#include <conio.h>
#include <string.h>
void main()
{
    char b[10],*c;
    int l,i;
    clrscr();
    cout<<"enter string";
```

```

cin>>b;

//c=&b[0];
c=b;
l=strlen(b);

cout<<"\n String is";
for(i=0;i<l;i++)
{
    cout<<*(c+i);
}

cout<<"\n Reverse string is ";
for(i=l-1;i>=0;i--)
{
    cout<<*(c+i);
}

getch();
}

```

4. Write a program to print a marksheet using an array of object.

```

#include <iostream.h>
#include <conio.h>

class std
{
    private:
        int r,t,p,m[5];
        char name[10];
    public:
        void getdata();
        void putdata();
}s[10];
void std::getdata()
{
    cout<<"enter r";
    cin>>r;
    cout<<"name";
    cin>>name;
    t=0;
    for(int i=0;i<5;i++)
    {
        cout<<"mark"<<i+1;
        cin>>m[i];
        t=t+m[i];
    }
    p=t/5;
}

```

```

void std::putdata()
{
    cout<<"marksheet"<<endl;
    cout<<"roll no"<<r<<endl;
    cout<<"name"<<name<<endl;
    for(int i=0;i<5;i++)
    {
        cout<<"marks"<<i+1<<" "<<m[i]<<endl;
    }
    cout<<"total"<<t<<endl;
    cout<<"per"<<p<<endl;
}
void main()
{
    clrscr();
    int n;
    cout<<"enter n";
    cin>>n;
    for(int i=0;i<n;i++)
    {
        s[i].getdata();
        s[i].putdata();
    }
    getch();
}

```

5. Write a program to take hours, minutes, and seconds, and display the total time in seconds.

```

#include <iostream.h>
#include <conio.h>

class time
{
private:
    int h,m,s,ts;
public:
    void get();
    void con();
    void put()
    {
        cout<<"hh:"<<h<<"mm:"<<m<<"ss:"<<s<<endl;
        cout<<"total second is"<<ts<<endl;
    }
};

void time::get()
{
    cout<<"enter h m s";
    cin>>h>>m>>s;
}

void time::con()

```

```
{  
    ts=0;  
    ts=h*3600+m*60+s;  
}  
void main()  
{  
    time t;  
    clrscr();  
    t.get();  
    t.con();  
    t.put();  
    getch();  
}
```

## Unit 2.Data Encapsulation and Inheritance

- 2.1 Access controls concepts (Public, Private, Protected) and differences among them.
  - 2.2 Declaring simple class, member variables and member functions.
  - 2.3 Concepts and use of enum.
  - 2.4 Concepts of Data hiding, abstraction and encapsulation with examples
  - 2.5 Concepts of Inheritance and Types of Inheritance
  - 2.6 Constructors and Destructors
- 

### 2.1 Access controls concepts (Public, Private, Protected) and differences among them.

Access modifiers are used to implement an important aspect of Object-Oriented Programming known as data hiding. There are three types of access modifiers available in C++.

1. Private
2. Public
3. Protected

If we do not specify any access modifiers for the members inside the class, then by default the access modifier for the members will be private.

**1. Public:** All class members declared under the public specifier will be accessible to everyone. The data members and member functions declared as public can be accessed by other classes and functions as well. The public members of a class can be accessed from anywhere in the program using the direct member access operator (.) with the object of that class.

#### Example 2.1 Public Access Modifier

```
#include <iostream.h>
#include <conio.h>

class circle
{
    public:
    int r;
    void area()
    {
        cout<<"area "<<3.14*r*r;
    }
};

void main()
{
    clrscr();
    circle c;
    cout<<"enter radius";
```

```

        cin>>c.r;
        c.area();
        getch();
    }

```

Output enter radius 10  
area 314

**2. Private:** The class members declared as private can be accessed only by the member functions inside the class. They cannot be accessed directly by any object or function outside the class. Only the member functions or friend functions are allowed to access the private data members of a class.

### Example 2.2 Private Access Modifier

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class circle
```

```
{
```

```
    private:
```

```
    int r;
```

```
    public:
```

```
    void area()
```

```
    {
```

```
        cout<<"enter radius";
```

```
        cin>>r;
```

```
        cout<<"area "<<3.14*r*r;
```

```
    }
```

```
};
```

```
void main()
```

```
{
```

```
    clrscr();
```

```
    circle c;
```

```
    c.area();
```

```
    getch();
```

```
}
```

Output enter radius 10  
area 314

**3. Protected:** The protected access modifier is similar to the private access modifier in that it can't be accessed outside of its class except with the help of a friend function. The difference is that the class members declared as protected can be accessed by any subclass (derived class) of that class as well.

**Example 2.3 Protected Access Modifier**

```
#include <iostream.h>
#include <conio.h>
class circle
{
    protected:
    int r;
    public:
    void area()
    {
        cout<<"enter radius";
        cin>>r;
        cout<<"area"<<3.14*r*r;
    }
};

void main()
{
    clrscr();
    circle c;
    c.area();
    getch();
}
```

Output enter radius 10  
area 314

**Example 2.4 Protected Access Modifier with Inheritance**

```
#include <iostream.h>
#include <conio.h>
class radius
{
    protected:
    int r;
};

class circle:public radius
{
    public:
    void area()
    {
        cout<<"enter radius";
        cin>>r;
        cout<<"area "<<3.14*r*r;
    }
};
```



```
void main()
{
    clrscr();
    circle c;
    c.area();
    getch();
}
```

Output enter radius 10  
area 314

## 2.2 Declaring simple class, member variables and member functions.

## 2.3 Concepts and use of enum.

An enumerated type (enumeration) is a user-defined data type that can be assigned a limited set of values. These values are defined by the programmer at the time of declaring the enumerated type.

It is also assigned starting from zero, similar to an array. Enumerations can also be used with switch statements.

**Syntax:** *enum enumerated-type-name{value1, value2, value3.....valueN};*

The enum keyword is used to declare enumerated types. After the enumerated type name is written, the possible values are defined within curly brackets. Once the enumerated type is defined, variables of that type can be created. There are two ways to create these variables.

1. They can be declared while declaring the enumerated type by adding the variable name before the semicolon.
2. Enumerated type variables can also be created later, just like normal variables.  
*enumerated-type-name variable-name = value;* By default, the starting value of the first element of an enum is 0 (similar to arrays), but this can be changed explicitly.

### Example 2.5 Enum example

```
#include <iostream.h>
#include <conio.h>

void main()
{
    // Defining enum Gender
    enum Gender { Male=1, Female }gen;

    // Creating Gender type variable
    Gender gender = Male;
    gen=Female;
    clrscr();

    cout<<gender<<endl;
    cout<<gen<<endl;
```

```

    getch();
}

```

```

Output: 1
        2

```

## 2.4 Concepts of Data hiding, abstraction and encapsulation with examples

**Encapsulation** is defined as wrapping up data and information into a single unit. In Object-Oriented Programming, encapsulation involves binding together the data and the functions that manipulate that data.

Consider a real-life example of encapsulation. In a company, there are different sections like the accounts section, finance section, sales section, etc. The finance section handles all financial transactions and keeps records of all the data related to finance. Similarly, the sales section handles all sales-related activities and keeps records of all sales.

Now, there may arise a situation where, for some reason, an official from the finance section needs all the data about sales for a particular month. In this case, he is not allowed to directly access the data of the sales section. He will first have to contact an officer in the sales section and then request the specific data. This is what encapsulation is. Here, the data of the sales section and the employees who can manipulate it are encapsulated under a single entity, the "sales section."

**Data abstraction** is one of the most essential and important features of object-oriented programming in C++. Abstraction means displaying only essential information and hiding the details. Data abstraction refers to providing only essential information about the data to the outside world while hiding the background details or implementation.

Consider a real-life example of a man driving a car. The man knows that pressing the accelerator will increase the speed of the car or applying the brakes will stop the car, but he does not know how pressing the accelerator actually increases the speed. He is unaware of the car's inner mechanisms or the implementation of the accelerator and brakes. This is what abstraction is.

## 2.5 Concepts of Inheritance and Types of Inheritance

The capability of a class to derive properties and characteristics from another class is called inheritance. Inheritance provides the concept of reusability. Classes can be reused through inheritance.

**Subclass:** The class that inherits properties from another class is called a subclass or derived class.

**Superclass:** The class whose properties are inherited by a subclass is called a base class or superclass.

```

Syntax: class subclass_name : access_mode base_class_name
{
    // body of subclass
}

```

```
};
```

The access modes are public, private, and protected. By default, the access mode is private.

### Access mode

**Public** - This is the most commonly used inheritance mode. In this mode, the protected members of the superclass become protected members of the subclass, and the public members remain public.

```
class DerivedClassName : public BaseClassName
```

**Private** - In private mode, the protected and public members of the superclass become private members of the derived class.

```
class DerivedClassName : private BaseClassName
```

**Protected** - In protected mode, the public and protected members of the superclass become protected members of the subclass.

```
class DerivedClassName : protected BaseClassName
```

|            | Derived Class | Derived Class | Derived Class  |
|------------|---------------|---------------|----------------|
| Base Class | Public Mode   | Private Mode  | Protected Mode |
| Private    | Not Inherited | Not Inherited | Not Inherited  |
| Protected  | Protected     | Private       | Protected      |
| Public     | Public        | Private       | Protected      |

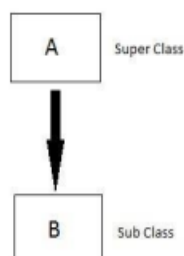
**Table of Visibility Modes**

### Types of Inheritance in C++

1. Single Inheritance
2. Multiple Inheritance
3. Hierarchical Inheritance
4. Multilevel Inheritance
5. Hybrid Inheritance (Virtual Inheritance)

#### 1. Single Inheritance

In this type of inheritance, one derived class inherits from only one base class. It is the simplest form of inheritance.



**Example 2.6 Single Inheritance**

```

#include <iostream.h>
#include <conio.h>
class A
{ public :
  int a;
  void getA()
  {cout<<"a"<<a<<endl; }
};
class B : public A
{ public :
  int b;
  void getB()
  { cout<<"b"<<b<<endl;}
};

void main()
{ B b1;
  clrscr();
  cout<<"enter a";
  cin>>b1.a;
  cout<<"enter b";
  cin>>b1.b;
  b1.getA();
  b1.getB();
  getch();
}

```

Output

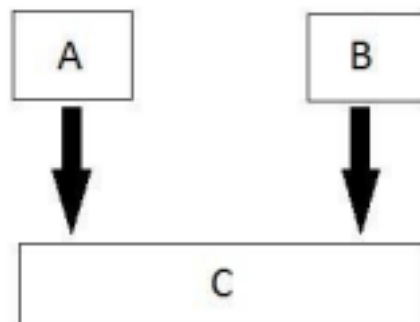
```

enter a1
enter b2
a1
b2

```

**2. Multiple Inheritance**

In this type of inheritance, a single derived class can inherit from two or more base classes.



**Example 2.7 Multiple Inheritance**

```

#include <iostream.h>
#include <conio.h>

class A
{
    public:
        int a;
        void geta()
        {
            cout<<"enter a";
            cin>>a;
        }
};

class B
{
    public:
        int b;
        void getb()
        {
            cout<<"enter b";
            cin>>b;
        }
};

class C:public A,public B
{
    public:
        void put()
        {
            cout<<"a "<<a<<" b "<<b;
        }
};

void main()
{
    C c;
    clrscr();
    c.geta();
    c.getb();
    c.put();
    getch();
}

```

Output

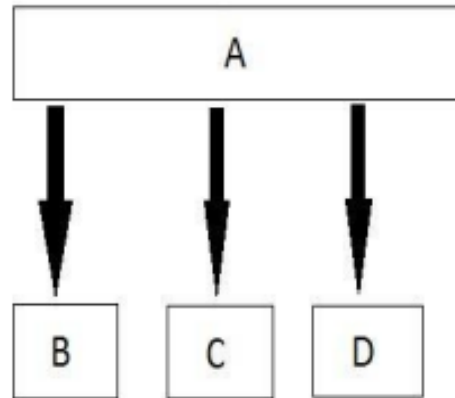
```

enter a1
enter b2
a 1 b 2

```

### 3.Hierarchical Inheritance

In this type of inheritance, multiple derived classes inherit from a single base class.



#### Example 2.8 Hierarchical Inheritance

```

#include <iostream.h>
#include <conio.h>

class A
{ public :
    int a;
    void getA()
    { cout<<" a "<<a<<endl; }
};

class B : public A
{ public :
    int b;
    void getB()
    { cout<<"b"<<b<<endl ; }
};

class C : public A
{ public :
    int c;
    void getC()
    { cout<<"c"<<c<<endl ; }
};

class D: public A
{ public :
    int d;
    void getD()
    { cout<<"d"<<d<<endl ; }
};

void main()
{ B b1;
  C c1;
  D d1;
  clrscr();
  cout<<"enter a";
  
```

```

cin>>b1.a;
cout<<"enter a";
cin>>c1.a;
cout<<"enter a";
cin>>d1.a;

cout<<"a from class b ";
b1.getA();
cout<<"a from class c ";
c1.getA();
cout<<"a from class d ";
d1.getA();

cout<<"enter b";
cin>>b1.b;
cout<<"enter c";
cin>>c1.c;
cout<<"enter d";
cin>>d1.d;

b1.getB() ;
c1.getC() ;
d1.getD() ;
getch();
}

```

Output :

```

enter a 1
enter a 2
enter a 3
a from class b a 1
a from class c a 2
a from class d a 3
enter b4
enter c5
enter d6
b4
c5
d6

```

#### 4. Multilevel Inheritance

In this type of inheritance, the derived class inherits from a class which, in turn, inherits from another class. The superclass for one is a subclass for the other.



**Example 2.9 Multilevel inheritance**

```

#include <iostream.h>
#include <conio.h>

class A
{ public :
  int a;
  void getA()
  { cout<<"a"<<a<<endl ; } };

class B : public A
{public :
  int b;
  void getB()
  { cout<<"b"<<b<<endl ; } };

class C : public B
{ public :
  int c;
  void getC()
  { cout<<"c"<<c ; } };

void main()
{ C c1;
  clrscr();
  cout<<"enter a";
  cin>>c1.a;
  cout<<"enter b";
  cin>>c1.b;
  cout<<"enter c";
  cin>>c1.c;
  c1.getA();
  c1.getB();
  c1.getC();
  getch();
}

```

**Output**

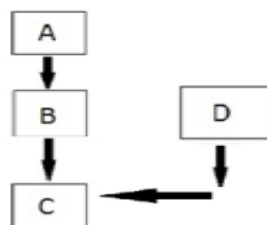
```

enter a 1
enter b 2
enter c 3
a1
b2
c3

```

**5. Hybrid Inheritance**

Hybrid inheritance is a combination of two or more types of inheritance.





**Example 2.10 Hybrid Inheritance**

```

#include <iostream.h>
#include <conio.h>

class A
{ public :
  int a;
  void getA()
  { cout<<"a"<<a<<endl; }
};

class D
{ public :
  int d;
  void getD()
  { cout<<"d"<<d<<endl; }
};

class B : public A
{ public :
  int b;
  void getB()
  { cout<<"b"<<b<<endl; }
};

class C : public B, public D
{ public :
  int c;
  void getC()
  { cout<<"c"<<c<<endl; }
};

void main()
{ C c1;
  clrscr();
  cout<<"enter a";
  cin>>c1.a;
  cout<<"enter b";
  cin>>c1.b;
  cout<<"enter c";
  cin>>c1.c;
  cout<<"enter d";
  cin>>c1.d;

  c1.getA();
  c1.getB();
  c1.getD();
  c1.getC();
  getch();
}

```

Output

```

enter a 1
enter b 2
enter c 3
enter d 4
a1
b2
d4
c3

```

## Constructor with inheritance

Base class constructors are always invoked in the constructors of derived classes. Whenever you create a derived class object, the default constructor of the base class is executed first, followed by the completion of the derived class's constructor.

In order to invoke the parameterized constructor of the base class within the parameterized constructor of the derived class, it must be explicitly specified during the declaration of the derived class's parameterized constructor.

### Example 2.11 Base class Constructor in Derived class Constructors

```

#include <iostream.h>
#include <conio.h>

class Base
{
    int x;
public:
    Base() // default constructor
    {
        cout << "Base default constructor\n";
    }

    Base(int i) // parameterized constructor
    { x=i;
      cout << "Base class parameterised constructor"<<x<<endl;
    }
};

class Derived : public Base
{
    int y;
public:
    Derived() // default constructor
    {
        cout << "Derived default constructor\n";
    }

    Derived(int i,int j):Base(j) // parameterized constructor

```

```

{ y=i;
cout << "Derived parameterized constructor"<<y<<endl;
}
};
void main()
{
clrscr();
Derived d1;
Derived d2(10,20);
getch();
}

```

Output

Base default constructor

Derived default constructor

Base class parameterised constructor 20

Derived parameterized constructor 10

### **Example 2.12 parameterised constructor in single inheritance**

```
#include <iostream.h>
```

```
#include <conio.h>
```

```

class A
{
public:
int a;
A(int x)
{
a=x;
}
void puta()
{
cout<<a;
}
};
class B:public A
{
public:
int b;
B(int x,int y):A(y)
{
b=x;
}
void putb()
{
cout<<b;
}
}

```

```
};
void main()
{
    clrscr();
    B b1(1,2);
    b1.puta();
    b1.putb();
    getch();
}
```

Output

21

### **Example 2.13 parameterised constructor in multiple inheritance**

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class A
```

```
{
    int a;
    public :
    A(int x)
    { a=x; }
    void putA()
    { cout<<a; }
};
```

```
class B
```

```
{
    int b;
    public :
    B(int x)
    {
        b=x;
    }
    void putB()
    { cout<<b; }
};
```

```
class C : public A ,public B
```

```
{
    int c;
    public :
    C(int x,int y,int z) : A(x),B(y)
    {
        c=z;
    }
    void putC()
    { cout<<c; }
};
```

```
};

int main()
{
    clrscr();
    C c1(1,2,3);
    c1.putA();
    c1.putB();
    c1.putC();
    getch();
    return(0);
}
```

Output

123

#### **Example 2.14 parameterized constructor in multilevel inheritance**

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class A
{
    int a;
public :
    A(int x)
    { a=x; }
    void putA() { cout<<a; }
};

class B : public A
{
    int b;
public :
    B(int x,int y):A(x)
    { b=y;}
    void putB() { cout<<b; }
};

class C : public B
{
    int c;
public :
    C(int x,int y,int z):B(x,y)
    { c=z; }
    void putC() { cout<<c; }
};

int main()
{
    clrscr();
    C c1(1,2,3);
    c1.putA();
```

```

    c1.putB();
    c1.putC();
    getch();
    return(0);
}

```

Output  
123

## 2.6 Constructors and Destructors

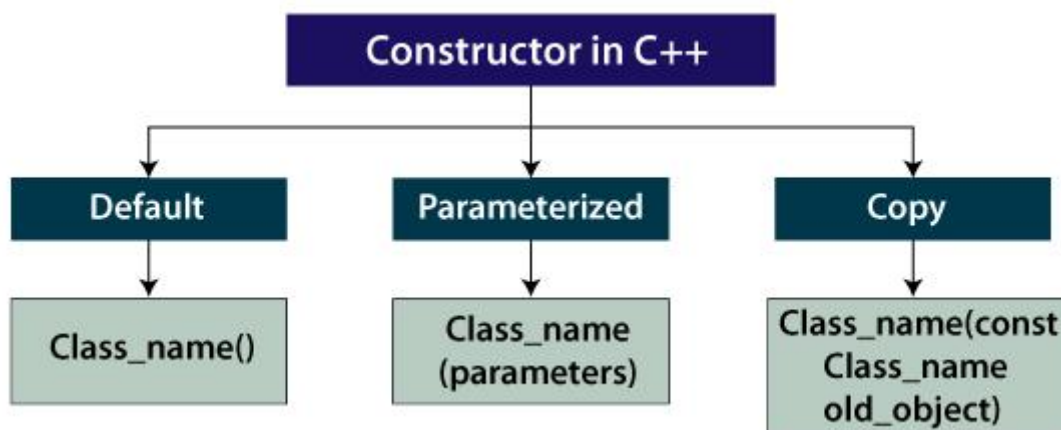
### What is a constructor?

A constructor is a member function of a class that initializes objects of the class. In C++, a constructor is automatically called when an object (instance of the class) is created. It is a special member function of the class.

### How are constructors different from a normal member function?

A constructor is different from a normal function in the following ways:

1. A constructor has the same name as the class itself.
2. Constructors don't have a return type.
3. A constructor is automatically called when an object is created.
4. If we do not specify a constructor, the C++ compiler generates a default constructor for us (which expects no parameters and has an empty body).



**Default Constructors:** A default constructor is a constructor that does not take any arguments and has no parameters.

### Example 2.15 Default constructor

```

#include <stdio.h>
#include <iostream.h>
#include <conio.h>

```

```

class construct
{
    int a, b;
public:
    construct() // Default Constructor
    {
        a = 10;
        b = 20;
    }
    void putdata() { cout<<a<<b; }
};

int main()
{
    // Default constructor called automatically
    // when the object is created
    construct c;
    clrscr();
    cout << "value of a:b ";
    c.putdata() ;
    getch();
    return(0);
}

```

Output:  
value of a:b1020

**Parameterized Constructors:** Parameterized Constructors allow for passing arguments to constructors, typically used to initialize an object when created. To create a parameterized constructor, add parameters to it and use these parameters in the constructor's body to initialize the object.

#### Example 2.16 Parameterized constructor

```

#include <iostream.h>
#include <conio.h>

class construct
{
    int a, b;
public:
    construct() { } // Default Constructor

    construct(int x,int y)//parameterized construcotor
    {
        a=x;
        b=y;
    }
}

```

```

void putdata() { cout<<a<<" "<<b; }
};

void main()
{
// when the object is created
clrscr();
construct c(5,10); //parameterized constructor
cout << "value of a:b ";
c.putdata() ;
getch();
}

```

Output:

value of a:b5 10

### Uses of Parameterized constructor:

1. It is used to initialise the various data elements of different objects with different values when they are created.
2. It is used to overload constructors.

**Copy Constructor:** The copy constructor is a member function that initializes an object using another object of the same class.

### Example 2.17 Copy constructor

```

#include <iostream.h>
#include <conio.h>

class Point
{
    private:
        int x, y;
    public:
        Point(int x1, int y1)//parameter constructor
        { x = x1; y = y1; }

        Point(Point &p) // Copy constructor
        { x = p.x;
          y = p.y;
        }
        void putdata() { cout<<x<<" "<<y<<endl; }
};

void main()
{
    Point p1(10, 15),p2(p1); /* Parameterized constructor is called here
    Point p2 = p1; Copy constructor is called here */
    clrscr();
}

```



```
p1.putdata();
p2.putdata();
getch();
}
```

Output

10 15

10 15

### Multiple Constructor in a class

When more than one constructor function is defined in a class, we say that the constructor is overloaded.

#### Example of different types of constructors.

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class test
```

```
{
```

```
    public:
```

```
        int a,b;
```

```
        test()
```

```
        {
```

```
            cout<<"enter a and b";
```

```
            cin>>a>>b;
```

```
        }
```

```
        test(int x,int y)
```

```
        {
```

```
            a=x;
```

```
            b=y;
```

```
        }
```

```
        test(test &t)
```

```
        {
```

```
            a=t.a;
```

```
            b=t.b;
```

```
        }
```

```
        void display()
```

```
        {
```

```
            cout<<a<<b<<endl;
```

```
        }
```

```
};
```

```
void main()
```

```
{
```

```
    clrscr();
```

```
    test t,t1(11,22),t2(t1);
```

```

        clrscr();
        cout<<"\n Value of a and b with Default constructor";
        t.display();
        cout<<"\n Value of a and b with parameterized constructor";
        t1.display();
        cout<<"\n Value of a and b with copy constructor";
        t2.display();
        getch();
    }

```

Output:

enter a and b1 2

Value of a and b with Default constructor12

Value of a and b with parameterized constructor1122

Value of a and b with copy constructor1122

### Default argument constructor

A default argument is a value provided in a function declaration that is automatically assigned by the compiler if the calling function doesn't provide a value for the argument. In case any value is passed, the default value is overridden.

#### Example of default argument constructor

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class num
```

```
{
```

```
    public:
```

```
    int a,b;
```

```
    num(int x,int y=1) // constructor with default argument
```

```
    {
```

```
        a=x;
```

```
        b=y;
```

```
    }
```

```
    void display()
```

```
    {
```

```
        cout<<"\na "<<a<<"b "<<b;
```

```
    }
```

```
};
```

```
void main()
```

```
{
```

```
    clrscr();
```

```
    num n(10);
```

```
    num n1(10,20);// implicit call
```

```

num n2=num(4,5); //explicit call
n.display();
n1.display();
n2.display();
getch();
}
Output:
a 10b 1
a 10b 20
a 4b 5

```

**What is a destructor?**

A destructor is a member function that destructs or deletes an object. A destructor function is called automatically when the object goes out of scope, which can happen when the function ends, the program ends, a block containing local variables ends, or a delete operator is called.

**How are destructors different from a normal member function?**

Destructors have the same name as the class preceded by a tilde (~).

**Example 2.18 Destructor**

```

#include <iostream.h>
#include <conio.h>
int c=0;
class set
{
public:
set()
{
c++;
cout<<"\n construct "<<c;
}
~set()
{
cout<<"\n destruct "<<c;
c--;
}
};
void main()
{
clrscr();
{
set a,b;
}
getch();
}

```

Output  
construct 1  
construct 2  
destruct 2  
destruct 1

## Unit 3.Polymorphism

- 3.1 Concepts of Polymorphism
  - 3.2 Compile time and Run time Polymorphism
  - 3.3 Overloading and Overriding
  - Concepts, difference and application
  - 3.4 Concepts of friend function
  - 3.5 Concepts of virtual function and pure virtual function
- 

### 3.1 Concepts of Polymorphism

### 3.2 Compile time and Run time Polymorphism

### 3.3 Overloading and Overriding. Concepts, difference and application

The word "polymorphism" means one name, multiple forms. In simple terms, we can define polymorphism as the ability of a message to be displayed in more than one form. A real-life example of polymorphism is a person who can exhibit different characteristics simultaneously. For instance, a man can be a father, a husband, and an employee at the same time. Thus, the same person possesses different behaviours in different situations. This is called polymorphism. Polymorphism is considered one of the important features of Object-Oriented Programming.

In C++ polymorphism is mainly divided into two types:

1. Compile time(Static) Polymorphism - Implemented using Function and Operator Overloading.
2. Runtime(Dynamic) Polymorphism- Implemented using Virtual Functions

#### Types of Polymorphism

1. Compile time polymorphism: This type of polymorphism is achieved by function overloading or operator overloading.

**Function Overloading** When there are multiple functions with the same name but different parameters, these functions are said to be overloaded. Functions can be overloaded by changing the number of arguments and/or the type of arguments.

#### Example 3.1 Function overloading

```
#include <iostream.h>
#include <conio.h>

class A
{
public:
void display(int a)
{
    cout<<a<<endl;
}
void display(double b)
{
    cout<<b<<endl;
```

```

    }
    void display(char *name)
    {
        cout<<name<<endl;
    }
};
void main()
{
    A a1;
    clrscr();
    a1.display(1);
    a1.display(1.2);
    a1.display("abc");
    getch();
}

```

Output

```

1
1.2
abc

```

## Operator Overloading

C++ allows us to add two variables of user-defined types using the same syntax that is applied to the basic types. This means that C++ has the ability to provide operators with a special meaning for a specific data type. This mechanism of giving a special meaning to an operator for a user-defined type is known as operator overloading.

Defining Operator Overloading:

To define an additional task for an operator, we must specify what it means in relation to the class to which the operator is applied. This is done with the help of a special function called an operator function.

General form of operator function is:

```

Return type classname : : operator op(argument list)
{
    Function body
}

```

Where return type is the type of value returned by function and op is the operator being overloaded. The op is preceded by the keyword operator. operator op is the function name.

Operator function must be either member function (non static) or friend functions.

The process of overloading involves the following steps:

1. Create a class that defines the data type that is to be used in the overloading operation.
2. Declare the operator function operator op () in the public part of the class. It may be either member or friend function.
3. Define the operator function.

Overloaded operator function can be call by expressions such as

**op x or x op** for unary operator. E.g.  $-x$  or  $x-$  where  $x$  is the object of class.

**x op y** for binary operator. E.g.  $x + y$  or  $x * y$  or  $x-y$  where  $x$  and  $y$  are objects of class.

Expression **x op y** would be interpreted as either

- **x . operator op( y )** in case of member function.
- **operator op( x, y )** in case of friend function.

Expression **op x or x op** would be interpreted as either

- **x . operator op( )** in case of member function.
- **operator op(x)** in case of friend function.

Rules for overloading operators

1. Only existing operators can be overloaded. New operator cannot be created.
2. The overloaded operator must have at least one operand that is of user defined type.
3. We cannot change the basic meaning of an operator. We cannot redefine the Plus (+) operator to subtract (-) one value from another.
4. Overloaded operators follow the syntax rules of the original operators. They cannot be overridden.
5. There are some operators that cannot be overloaded.
 

|        |                            |
|--------|----------------------------|
| Sizeof | Size of operator           |
| .      | Membership operator        |
| .*     | Pointer to member operator |
| ::     | Scope resolution operator  |
| ?:     | Conditional operator       |
6. We cannot use the friend function to overload certain operators. However member functions can be used to overload them.
 

|    |                              |
|----|------------------------------|
| =  | assignment operator          |
| () | function call operator       |
| [] | subscripting operator        |
| -> | class member access operator |
7. Unary operator overloaded by member function, take no explicit arguments and return no explicit values. But those overloaded by a friend function take one explicit argument (object of the relevant class).
8. Binary operator overloaded by member function, take one explicit argument and those overloaded by a friend function take two explicit arguments.
9. When using a binary operator overloaded through a member function the left hand operand must be an object of the relevant class.
10. Binary arithmetic operators such as  $+$ ,  $-$ ,  $*$ ,  $/$  must explicitly return a value.

### Example 3.2 Program for Unary (-) and Binary (+) Operator Overloading Using a Member Function

```
#include <iostream.h>
#include <conio.h>
```

```
class data
{
```

```

private:
    int a,b;
public:
    data()
    {
        a=0;
        b=0;
    }
    data(int x,int y)
    {
        a=x;
        b=y;
    }
    data operator+(data d2)//binary operator +
    {
        data d;
        d.a=a+d2.a;
        d.b=b+d2.b;
        return(d);
    }
    void operator-()//unary - operator
    {
        a=-a;
        b=-b;
    }
    void display()
    {
        cout<<"a= "<<a<<endl<<"b= "<<b<<endl;
    }
};

void main()
{
    clrscr();
    data d1(-1,-1),d2(2,2),d3;
    d1.operator-();
    //-d1;
    d1.display();
    d3=d1.operator+(d2);
    //d3=d1+d2;
    d3.display();
    getch();
}

```

Output

```

1
1
3
3

```



**Example 3.3 Program for Unary (-) and Binary (+) Operator Overloading Using a Friend Function**

```

#include <iostream.h>
#include <conio.h>
#include <string.h>
class data
{
    private:
        int a,b;
    public:
        data()
        {
            a=0;
            b=0;
        }
        data(int x,int y)
        {
            a=x;
            b=y;
        }

        friend data operator+(data d1,data d2)
        {
            data d;
            d.a=d1.a+d2.a;
            d.b=d1.b+d2.b;
            return(d);
        }
        friend void operator-(data &d1)
        {
            d1.a=-d1.a;
            d1.b=-d1.b;
        }
        void display()
        {
            cout<<a<<b<<endl;
        }
};

void main()
{
    clrscr();
    data d3,d1(-1,-1),d2(2,2);
    operator-(d1);
    //-d1;
    d1.display();
    d2.display();
    d3=operator+(d1,d2);

```

```
//d3=d1+d2;
d3.display();
getch();
}
```

Output

```
11
22
33
```

### 3.4 Concepts of friend function

According to the data hiding concept in C++, no one has permission to access private and protected members from outside the class. If a developer needs access to these private or protected members, they can use friend functions. A friend function provides the privilege to access all the private and protected members of the class.

#### Friend Function Characteristics

1. It should be declared inside the class with the 'friend' keyword.
2. It can be defined anywhere in the program and outside of the class like a normal function, but the friend keyword must not be used in the function definition.
3. It is not within the class scope, so it cannot be called by objects of the class.
4. It can be called like a normal function.
5. Usually, it has class objects as arguments.
6. It cannot access member variables or functions directly; it can only access them with the help of the class objects using the dot membership operator.
7. It can be declared with any access specifier, and the access specifier does not impact the friend function.

#### Example 3.4 Write a program to find the mean of numbers using a friend function.

```
#include<iostream.h>
#include<conio.h>
class A
{
public:
    A()
    {
        cout<<"enter a and b";
        cin>>a>>b;
    }
private:
    int a,b;
    friend float mean(A);
```

```

};

float mean(A a1)
{
    return float(a1.a+a1.b)/2.0;
}
void main()
{
    clrscr();
    A a1;
    cout<<"mean is"<<mean(a1);
    getch();
}
Output
enter a and b1 2
mean is1.5

```

### Passing and returning object

In C++ we can pass class's objects as arguments and also return them from a function the same way we pass and return other variables.

Syntax `object_type function_name(object);`

Example: Write a program to add two time objects and display the sum using the friend function.

```

//object as argument and return
//read and add time
#include<iostream.h>
#include<conio.h>

class time
{
    int hours;
    int min;
public:
    void gettime(int h,int m)
    {
        hours=h;
        min=m;
    }
    void puttime()
    {
        cout<<"\n\n\t Hours :- "<<hours;
        cout<<"\n\n\t Minutes :- "<<min;
    }
    friend time sum(time,time);
}

```

```

};
time sum(time t1,time t2)
{
    time t;
    t.min=t1.min+t2.min;
    t.hours=t.min/60;
    t.min=t.min%60;
    t.hours=t.hours+t1.hours+t2.hours;
    return(t);
}

void main()
{
    time t1,t2,t3;
    clrscr();
    t1.gettime(1,40);
    t2.gettime(1,30);
    t3=sum(t1,t2);
    cout<<"\t\t\t TIME T1 :- ";t1.puttime();
    cout<<"\n\t\t\t TIME T2 :- ";t2.puttime();
    cout<<"\n\t\t\t THE ADDITION OF TWO TIMES ";
    cout<<"\n\n\t\t\t TIME T3 :- ";t3.puttime();
    getch();
}

```

Output:

```

                                TIME T1:-
Hours:- 1
Minutes:- 40

                                TIME T2:-
Hours:- 1
Minutes:- 30

                                THE ADDITION OF TWO TIMES
                                TIME T3:-
Hours:- 3
Minutes:- 10

```

### 3.5 Concepts of virtual function and pure virtual function

A virtual function is a member function declared within a base class and redefined (overridden) by a derived class. When you refer to a derived class object using a pointer or a reference to the base class, you can call a virtual function for that object and execute the derived class's version of the function. Virtual functions ensure that the correct function is called for an object, regardless of the type of reference (or pointer) used for the function call. They are mainly used to achieve runtime polymorphism. Functions are declared with the virtual keyword in the base class. The resolving of function calls is done at runtime.

Rules for Virtual Functions:

1. Virtual functions cannot be static.
2. A virtual function can be a friend function of another class.
3. Virtual functions should be accessed using a pointer or reference of the base class type to achieve runtime polymorphism.
4. The prototype of virtual functions should be the same in the base as well as derived class.
5. They are always defined in the base class and overridden in a derived class. It is not mandatory for the derived class to override (or redefine) the virtual function; in that case, the base class version of the function is used.
6. A class may have a virtual destructor, but it cannot have a virtual constructor.

**Example 3.5 virtual function**

```
#include <iostream.h>
#include <conio.h>

class base
{
public:
virtual void print()
{
cout << "print base class" << endl;
}
void show()
{
cout << "show base class" << endl;
}
};

class derived : public base
{
public:
void print()
{
cout << "print derived class" << endl;
}
void show()
{
cout << "show derived class" << endl;
}
};

int main()
{
base *bptr;
derived d;
bptr = &d;
clrscr();
// virtual function, binded at runtime
bptr->print();
// Non-virtual function, binded at compile time
bptr->show();
```

```

getch();
return(0);
}

```

Output

print derived class

show base class

A virtual function is a member function declared within a base class and is redefined (overridden) by a derived class. When you refer to a derived class object using a pointer or a reference to the base class, you can call a virtual function for that object and execute the derived class's version of the function.

Example virtual void print() {cout<<"hello";}

Pure Virtual Functions in C++:

A pure virtual function (or abstract function) in C++ is a virtual function for which we don't have an implementation; we only declare it. A pure virtual function is declared by assigning 0 in the declaration.

Example virtual void print()=0;

### **Programming exercises and solutions**

1. Write a program to find the area of a circle and a rectangle using function overloading.

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class test
```

```
{
```

```
    public:
```

```
        void area(double r)
```

```
        {
```

```
            cout<<"area of circle is "<<3.14*r*r<<endl;
```

```
        }
```

```
        void area(float l,float b)
```

```
        {
```

```
            cout<<"area of rect is "<<l*b<<endl;
```

```
        }
```

```
};
```

```
void main()
```

```
{
```

```
    test t;
```

```
    clrscr();
```

```
    t.area(10);
```

```
    t.area(1.5,1.5);
```

```
    getch();
```

```
}
```

2. operator overload + to concat two strings and == to compare strings.

```
#include <iostream.h>
#include <conio.h>
#include <string.h>
class data
{
    private:
        char *a;
    public:
        data()
        {
            a=0;//null string;
        }
        data(char *s)
        {
            strcpy(a,s);
        }

        data operator+(data d2)
        {
            data d3;
            strcpy(d3.a,a);
            return(strcat(d3.a,d2.a));
        }
        void show()
        {
            cout<<a;
        }
        int operator==(data d2)
        {
            if(strcmp(a,d2.a)==0)
                return(1);
            else
                return(0);
        }
};

void main()
{
    clrscr();
    char s1[10],s2[10];
    cin>>s1>>s2;
    data d1(s1),d2(s2),d3;
    //d3=d1.operator+(d2);
    d3=d1+d2;
    d3.show();
    if(d1==d2)
        cout<<"\n same";
```

```

else
    cout<<"\n not same";
    getch();
}

```

3.operator overload > to find a large number from two numbers.

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
#include <string.h>
```

```
class data
```

```
{
```

```
private:
```

```
int a;
```

```
public:
```

```
data()
```

```
{
```

```
    cin>>a;
```

```
}
```

```
int operator>(data d2)
```

```
{
```

```
    if(a>d2.a)
```

```
        return(1);
```

```
    else
```

```
        return(0);
```

```
}
```

```
void show()
```

```
{
```

```
    cout<<a;
```

```
}
```

```
};
```

```
void main()
```

```
{
```

```
    clrscr();
```

```
    data d1,d2;
```

```
    if(d1>d2)
```

```
{
```

```
        d1.show();
```

```
        cout<<" is large";
```

```
}
```

```
else
```

```
{
```

```
        d2.show();
```

```
        cout<<" is large";
```

```
}
```

```
    getch();
```

```
}
```



4. Write a program to overload + operator for one dim matrix.

```
#include <iostream.h>
#include <conio.h>

class data
{
    private:
        int a[5];
    public:
        void get();
        data operator+(data);
        void display();
};

void data::get()
{
    for(int i=0;i<5;i++)
        cin>>a[i];
}

void data:: display()
{
    for(int i=0;i<5;i++)
        cout<<a[i]<<endl;
}

data data::operator+(data d2)
{
    data d;
    for(int i=0;i<5;i++)
        d.a[i]=a[i]+d2.a[i];
    return(d);
}

void main()
{
    clrscr();
    data d,d1,d2;
    d1.get();
    d2.get();
    clrscr();
    d=d1+d2;
    d.display();
    getch();
}
```

5. Create a base class shape, declare two data members dim1 and dim2 and declare- calculate area function as virtual function. Derive two class rectangle and triangle from the shape class that override calculate area function.

```

#include<iostream.h>
#include<conio.h>
class shape
{
    protected:
        int d1,d2;
    public:
        virtual void area()=0;
};
class rect:public shape
{
    public:
        rect(int x,int y)
        {
            d1=x;
            d2=y;
        }

        void area()
        {
            cout<<"\n area of rectangle"<<d1*d2;
        }
};
class triangle:public shape
{
    public:
        triangle(int x,int y)
        {
            d1=x;
            d2=y;
        }

        void area()
        {
            cout<<"\n area of triangle "<<0.5*d1*d2;
        }
};

void main()
{
    clrscr();
    shape *s1;
    rect r(10,10);
    triangle t(20,20);
    s1=&r;
    s1->area();
    s1=&t;
    s1->area();
    getch();
}

```

```
}

```

### Output

```
area of rectangle 100
```

```
area of triangle 200
```

6. Write a program to create a base class media with data member (title, price). derive tape and book from the media. Declare virtual function display in media and display book detail (title, price, page) and tape detail (title, price, playtime) using display function in respective classes.

```
#include<iostream.h>
#include<conio.h>
#include<string.h>
class media
{
    protected:
    char title[50];
    float price;
    public:
    media(char *s, float a)
    {
        strcpy(title, s);
        price = a;
    }

    virtual void display() = 0;
};

class tape : public media {
public:
    float time;
    tape(char *s, float a, float t) : media(s, a)
    {
        time = t;
    }
    void display()
    {
        cout << "\n Title" << title;
        cout << "\n Prices" << price;
        cout << "\n Play time" << time;
    }
};

class book : public media {
public:
    int pages;
    book(char *s, float a, int p) : media(s, a)
```

```

        {
            pages=p;
        }
void display()
{
    cout << "\n Title"<<title;
    cout << "\n Prices"<<price;
    cout << "\n Pages"<<pages;
}
};
void main()
{
    char *t;
    float p,pt;
    int pg;

    clrscr();
    //Book detail
    cout<<"\n enter book detail\n";
    cout<<"Title ";cin>>t;
    cout<<"Prices ";cin>>p;
    cout<<"Pages ";cin>>pg;

    book bk(t,p,pg);

    //Tape detail
    cout<<"\n enter tape detail\n";
    cout<<"Title ";cin>>t;
    cout<<"Prices ";cin>>p;
    cout<<"Play time ";cin>>pt;

    tape tp(t,p,pt);

    clrscr();
    media *list[2];
    list[0]=&bk;
    list[1]=&tp;

    cout<<"\n Book Detail";
    list[0]->display(); // call book display fun

    cout<<"\n\n Tape Detail";
    list[1]->display();

    getch();
}

```

## Unit 4. Data Structure

4.1 Introduction of Data Structure and application areas.

4.2 Recursion concepts

4.3 Difference among Linear and Non-Linear Data Structure

4.4 Stack

- Concepts of Stack(LIFO)
- Pop, Push and Display(Peep)
- Application areas of Stack  
( Infix to postfix, Infix to prefix )

---

### 4.1 Introduction of Data Structure and application areas.

A data structure is a storage system used to store and organise data. It arranges data on a computer in a way that allows for efficient access and updates.

Depending on your requirements and project, it is important to choose the right data structure. For example, if you want to store data sequentially in memory, you can use the array data structure.

The following are some of the most important areas where data structures are used:

Compiler design

Machine learning

Database design and management

Numerical and Statistical analysis

Operating system development

Types of Data Structures:

Data structures are basically divided into two categories:

1. Linear data structures
2. Non-linear data structures

#### Linear data structures

In linear data structures, the elements are arranged in sequence, one after the other. Since the elements are arranged in a particular order, they are easy to implement. However, as the complexity of the program increases, managing these structures can become challenging.

The linear data structures are:

- Arrays
- Queues
- Stacks
- Linked lists

#### Non linear data structures

Unlike linear data structures, elements in non-linear data structures are not arranged in a sequence. Instead, they are organised in a hierarchical manner where one element is connected to one or more elements.

The non-linear data structures are:

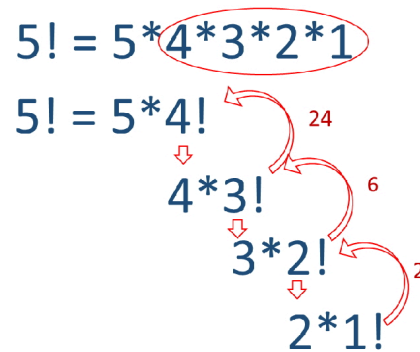
- Graphs
- Trees

## 4.2 Recursion concepts

This function calls itself repeatedly. It must have an exit condition that must be satisfied; otherwise, the recursive function will call itself indefinitely.

### Recursive Algorithm for Factorial

1. **Base Case:**
  - a. If  $n$  is 1, return 1. ( $1!=1$ )
2. **Recursive Case:**
  - a. For any other positive integer  $n$ , return  $n$  multiplied by the factorial of  $n-1$ . (i.e.  $n!=n \times \text{factorial}(n-1)!$ )



**Example 4.1 Write a program to find the factorial using a recursive function.**

```
#include <iostream.h>
#include <conio.h>
class rec
{
    public:
        int fact(int);
};
int rec::fact(int n)//function definition
{
    if(n==1)
        return 1;
    else
        return n*fact(n-1);//recursive call
}

void main()
{
    rec r;
    int x=5;
    clrscr();
    cout<<"fact of "<<x<<" is "<<r.fact(x);//function call
    getch();
}
```

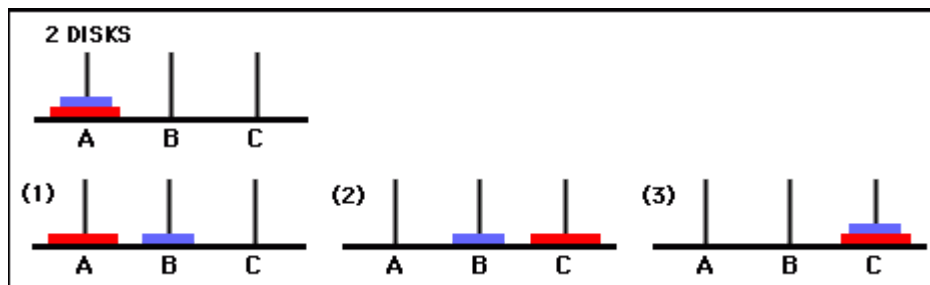
Output  
fact of 5 is 120

### Tower of Hanoi

The Tower of Hanoi is a well-known problem in computer science and can be solved using a recursive algorithm. It involves three rods and a number of disks of different sizes arranged in ascending order. The objective is to move the entire disk to another rod, following these rules:

1. Only one disk can be moved at a time.
2. Each move consists of taking the upper disk from one of the rod and placing it on top of another rod or on an empty rod.
3. No disk may be placed on top of a smaller disk.

Example with N=2 disk.



### Algorithm

**1. Base Case:** If there is only one disk, move it from the source rod to the destination rod.

**2. Recursive Case:**

- Move the top (n-1) disks from the source rod to the auxiliary rod, using the destination rod as a temporary holding area.
- Move the nth disk (the largest disk) directly from the source rod to the destination rod.
- Move the (n-1) disks from the auxiliary rod to the destination rod, using the source rod as a temporary holding area.

#### Example 4.2 Tower of Hanoi implementation

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
void towerOfHanoi(int n, char a, char c, char b)
```

```
{
```

```
    if (n == 1)
```

```
    {
```

```
        cout<<" Move disk "<<n<<" from rod "<<a<<" to rod "<<c<<"\n";
```

```
        return;
```

```
    }
```

```

towerOfHanoi(n - 1,a,b,c);
cout << " Move disk " <<n<< " from rod " <<a<< " to rod " <<c<<"\n";
towerOfHanoi(n - 1,b,c,a);
}

int main()
{
    int N;
    clrscr();
    cout<<"\n enter n";
    cin>>N;
    // A, B and C are names of rods
    towerOfHanoi(N, 'A', 'C', 'B');
    getch();
    return 0;
}

```

Output :

enter n2

Move disk 1 from roa A to rod B

Move disk 2 from roa A to rod C

Move disk 1 from roa B to rod C

### 4.3 Difference among Linear and Non-Linear Data Structure

| Linear Data Structures                                                                                                                            | Non Linear Data Structures                                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| The data items are arranged in sequential order, one after the other.                                                                             | The data items are arranged in non-sequential order (hierarchical manner).                                                                     |
| All the items are present on the single layer.                                                                                                    | The data items are present at different layers.                                                                                                |
| It can be traversed on a single run. That is, if we start from the first element, we can traverse all the elements sequentially in a single pass. | It requires multiple runs. That is, if we start from the first element it might not be possible to traverse all the elements in a single pass. |
| The memory utilisation is not efficient.                                                                                                          | Different structures utilise memory in different efficient ways depending on the need.                                                         |
| The time complexity increases with the data size.                                                                                                 | Time complexity remains the same.                                                                                                              |
| Example: Arrays, Stack, Queue                                                                                                                     | Example: Tree, Graph, Map                                                                                                                      |

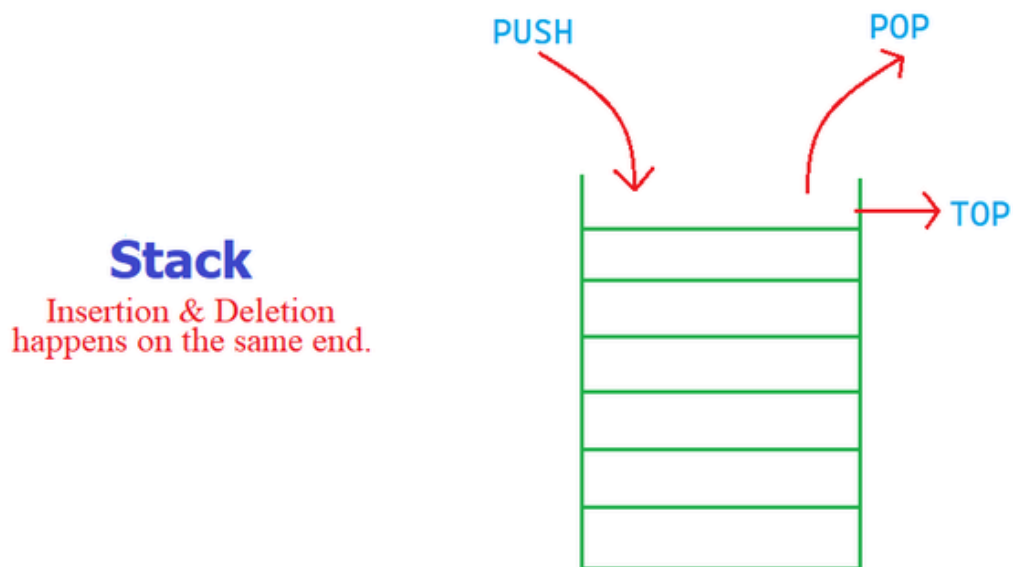
### 4.4 Stack

#### - Concepts of Stack(LIFO)

#### - Pop, Push and Display(Peep)

A stack is a linear data structure that follows a particular order for performing operations. The order may be LIFO (Last In First Out) or FILO (First In Last Out).





There are many real-life examples of a stack. Consider the simple example of plates stacked on top of one another in a canteen. The plate on the top is the first one to be removed, i.e., the plate placed at the bottom remains in the stack for the longest period of time. This clearly demonstrates the LIFO/FILO order. The following four basic operations are mainly performed in a stack:

### Stack Operations

The basic four operations of a stack are push, pop, peep, and change(UPDATE).

- 1) Push - Inserts an element X at the top of a stack represented by array S, which contains N elements, with a pointer TOP indicating the current top element of the stack.

Step 1: [ check stack overflow]

    If  $TOP \geq N$  then

        write 'stack overflow'

        return

Step 2: [Increment top]

$TOP = TOP + 1$

Step 3: [Insert element]

$S[TOP] = X$

Step 4: [finished]

    return

- 2) POP - removes the top element from a stack represented by the array S and returns that element. The pointer TOP indicates the current top element of the stack.

Step 1: [Check for underflow]

    If  $TOP == 0$  then

        write 'stack underflow'

        return

Step 2: [ Decrement TOP]

TOP = TOP -1

Step 3: [ return element of stack]  
return(S[TOP+1])

3) PEEP - return the value of the ith element from the top of the stack.

Step 1: [check for underflow]  
If TOP-i+1 <=0 then  
write ' stack underflow on peep'  
return

Step 2: [ return i th element from top of stack]  
return(s[TOP-i+1])

4) CHANGE - Updates the value of the ith element from the top of the stack to the value stored in X.

Step 1: [check for underflow]  
If TOP-i+1 <=0 then  
write ' stack underflow on change'  
return

Step 2: [change i th element from top of stack]  
s[TOP-i+1]=X

#### Example 4.3 Stack implementation

```
#include<iostream.h>
#include<conio.h>
#include<stdlib.h>
class stack
{
int s[4];
int top;
public:
stack()
{top = 0;}
void push(int x)
{
if(top >= 3)
{
cout << "stack overflow";
return;
}
top=top+1;
s[top] = x;
cout << "inserted " << x;
}
void pop()
{
if(top == 0)
```

```

        {
            cout << "stack underflow";
            return;
        }
        cout << "deleted " << s[top--];
    }

void peep();
void change();
void display();
};

void stack::peep()
{
    int i,t;
    cout<<"enter the position";
    cin>>i;

    if(top-i+1<= 0)
    { cout << " stack underflow on peep"; return;}

    cout<<s[top-i+1];
}
void stack::change()
{
    int i,item;
    cout<<"\n enter the position to enter";
    cin>>i;
    cout<<"\n enter the item to be insert :";
    cin>>item;

    if(top-i+1<= 0)
    { cout << " stack underflow on change";return;}

    s[top-i+1]=item;
}

void stack::display()
{
    if(top == 0)
    {
        cout << " stack underflow";
        return;
    }
    for(int i = top; i >= 1; i--)
        cout <<"| s["<<i<<"] |"<< s[i] << "| \n";
}

int main()

```

```

{
int ch;
stack st;
clrscr();
while (1)
{
clrscr();
cout << "\n 1.push";
cout << "\n 2.pop";
cout << "\n 3.display";
cout << "\n 4.peep";
cout << "\n 5.change";
cout << "\n 6.exit";
cout << "\n enter ur choice: ";
cin >> ch;
switch (ch)
{
case 1:
cout << "enter the element: ";
cin >> ch;
st.push(ch);
break;
case 2:
st.pop();
break;
case 3:
st.display();
break;
case 4:
st.peep();
break;
case 5:
st.change();
break;
case 6:
exit(0);
}
getch();
}
}

```

### Application areas of Stack

( Infix to postfix, Infix to prefix )

#### Infix to postfix

**Infix expression:** An expression of the form 'a op b' has an operator between every pair of operands. For example, 'a + b'.

**Postfix expression:** An expression of the form 'a b op' has an operator following every pair of operands. For example, 'ab+'.

**Infix Expression :  $A+(B*C+D)/E$** 

Push "(" into stack and add "(" to the end of the infix expression.

| Input Token | Stack | Postfix Expression | Action                                                                                                                  |
|-------------|-------|--------------------|-------------------------------------------------------------------------------------------------------------------------|
|             | (     |                    | Add '(' to the stack.                                                                                                   |
| A           | (     | A                  | Add A to the postfix expression.                                                                                        |
| +           | (+    | A                  | Push '+' onto the stack.                                                                                                |
| (           | (+(   | A                  | Push '(' onto the stack.                                                                                                |
| B           | (+(   | AB                 | Add B to the postfix expression.                                                                                        |
| *           | (+(*  | AB                 | Push '*' onto the stack.                                                                                                |
| C           | (+(*  | ABC                | Add C to the postfix expression.                                                                                        |
| +           | (+(+  | ABC*               | '*' operator has higher precedence than '+', so pop * and add it to the postfix expression. Also, add '+' to the stack. |
| D           | (+(+  | ABC*D              | Add D to the postfix expression.                                                                                        |
| )           | (+    | ABC*D+             | ' )' has come so pop + and add it to the postfix expression. Also, remove '(' from the stack.                           |
| /           | (+(/  | ABC*D+             | '+' has lower precedence than '/', so push '/' onto the stack.                                                          |
| E           | (+(/  | ABC*D+E            | Add E to the postfix expression.                                                                                        |
| )           |       | ABC*D+E/+          | ' )' has come, so pop all operators one by one from the stack and add them to the postfix expression.                   |

**Algorithm**

Step 1: Push "(" onto stack and add "(" to the end of the infix expression.

Step 2: Read the infix expression(left to right).Scan the characters one by one.

Step 3: Repeat until each character in the infix notation has been scanned.

If a "(" is encountered, push it onto the stack.

If an operand (whether a digit or a character) is encountered, add it to the postfix expression.

If a ")" is encountered, then

- Repeatedly pop operators from the stack and add them to the postfix expression until a "(" is encountered.
- Discard the "(" . Also remove the "(" from stack and do not add it to the postfix expression.

If an operator is encountered, then

- Repeatedly pop operators from the stack and add each operator (popped from the stack) to the postfix expression if it has the same precedence or a higher precedence than the current operator.
- Push the operator onto the stack.

[END OF IF]

[END OF LOOP]

Step 4: EXIT

#### Example 4.4 Infix to postfix expression

```
#include <iostream.h>
#include <ctype.h>
#include <conio.h>
#include <string.h>
#define SIZE 50

char stack[SIZE];
int top=-1;    /* Global declarations */
class postf
{
public:
void push(char elem)
{
    /* Function for PUSH operation */
    stack[++top]=elem;
}

char pop()
{
    /* Function for POP operation */
    return(stack[top--]);
}

int pr(char symbol)
{
    /* Function for precedence */

    if(symbol == '^')/* exponent operator, highest precedence*/
    {
        return(3);
    }
    else if(symbol == '*' || symbol == '/')
    {
        return(2);
    }
    else if(symbol == '+' || symbol == '-')    /* lowest precedence */
    {
        return(1);
    }
}
```

```

        else
        {
            return(0);
        }
    }
};
/* Main Program */
void main()
{
    char infix[50],postfix[50],ch;
    int i=0,k=0;
    postf p;
    clrscr();
    cout<<"Enter Infix Expression : ";
    cin>>infix;
    p.push('(');
    strcat(infix,"");
    while((ch=infix[i++]) != '\0')
    {
        if( ch == '(') p.push(ch);
        else if(isalnum(ch)) postfix[k++]=ch;
        else if( ch == ')')
        {
            while( stack[top] != '(')
                postfix[k++]=p.pop();
            p.pop(); /* Remove ( */
        }
        else
        { /* Operator */
            while(p.pr(stack[top]) >=p.pr(ch) )
                postfix[k++]=p.pop();
            p.push(ch);
        }
    }
    postfix[k]='\0'; /* Make postfix as valid string */
    cout<<"\n Postfix Expression = "<<postfix;
    getch();
}
output :
Enter Infix Expression :a+b
Postfix Expression = ab+

```

## Infix to prefix

**Infix expression:** An expression of the form 'a op b' has an operator between every pair of operands. For example, 'a + b'.

**Prefix expression:** An expression of the form 'op a b' has an operator followed by operands. For example, +ab.

**Infix Expression:** (A+B)+C-(D-E)^F

Push “)” into the stack. then reverse the given infix expression(scan from right to left) and add “(” to the end of the infix expression.

**After Reversing String is:** F^)E-D(-C+)B+A(

| Input Token | Stack | Expression  | Action                                                                                                                                |
|-------------|-------|-------------|---------------------------------------------------------------------------------------------------------------------------------------|
|             | )     |             | Add ')' to the stack.                                                                                                                 |
| F           | )     | F           | Add F to the expression string..                                                                                                      |
| ^           | )^    | F           | Push '^' onto the stack.                                                                                                              |
| )           | )^)   | F           | Push ')' onto the stack.                                                                                                              |
| E           | )^)   | FE          | Add E to the expression string.                                                                                                       |
| -           | )^)-  | FE          | Push '-' onto the stack.                                                                                                              |
| D           | )^)-  | FED         | Add D to the expression string.                                                                                                       |
| (           | )^    | FED-        | '(' pair matched, so pop the operator '-' and add it to the expression string. Also, remove ')' from the stack.                       |
| -           | )-    | FED-^       | '^' operator has higher precedence than '-', so pop '^' and add it to the expression string. Also, add the '-' operator to the stack. |
| C           | )-    | FED-^C      | Add C to the expression string.                                                                                                       |
| +           | )+    | FED-^C-     | Pop '-' because '-' and '+' have the same precedence. Also, add the '+' operator to the stack.                                        |
| )           | )++)  | FED-^C-     | Push ')' onto the stack.                                                                                                              |
| B           | )++)  | FED-^C-B    | Add B to the expression string.                                                                                                       |
| +           | )++)+ | FED-^C-B    | Push '+' onto the stack.                                                                                                              |
| A           | )++)+ | FED-^C-BA   | Add A to the expression string.                                                                                                       |
| (           | )+    | FED-^C-BA+  | '(' pair matched, so pop the operator '+' and add it to the expression string. Also, remove ')' from the stack.                       |
| (           | )+    | FED-^C-BA++ | '(' pair matched, so pop all operators one by one as we have reached the end of the expression.                                       |



Now **reverse** the expression to get prefix expression ++AB-C^--DEF

### Algorithm

1. Push "(" onto the stack. Then, reverse the given infix expression(scan from right to left) and add "(" to the end of the infix expression.
2. Scan the characters one by one.
3. Repeat until each character in the infix notation has been scanned.  
If the character is an operand, add it to the prefix expression.

If the character is a closing parenthesis, push it onto the stack.

If the character is an opening parenthesis

- pop the elements from the stack until we find the corresponding closing parenthesis.
- Discard the ")". Also remove the "(" from stack and do not add it to the prefix expression.

If the character scanned is an operator

- Repeatedly pop from stack and add each operator (popped from the stack) to the prefix expression if it has the same precedence or a higher precedence than the current operator.
- Push the operator onto the stack.

[END OF IF]

[END OF LOOP]

4. After all the characters are scanned, reverse the prefix notation.

#### Expression 4.5 Infix to prefix expression

```
#define SIZE 50          /* Size of Stack */
#include <iostream.h>
#include <conio.h>
#include <string.h>
#include <ctype.h>
char s[SIZE];
int top=-1; /* Global declarations */
class pref
{
public:
void push(char elem)
{
    /* Function for PUSH operation */
    s[++top]=elem;
}
char pop()
{
    /* Function for POP operation */
    return(s[top--]);
}
int pr(char symbol)
{
    /* Function for precedence */

    if(symbol == '^')/* exponent operator, highest precedence*/
    {
        return(3);
    }
}
```

```

        else if(symbol == '*' || symbol == '/')
        {
            return(2);
        }
        else if(symbol == '+' || symbol == '-')    /* lowest precedence */
        {
            return(1);
        }
        else
        {
            return(0);
        }
    }
};

void main()
{
    /* Main Program */
    char infix[50],prfx[50],ch;
    int i=0,k=0;
    pref p;
    clrscr();
    cout<<"\n\nInfix Expression";
    cin>>infix;
    p.push('(');
    strrev(infix);
    strcat(infix,"(");
    while( (ch=infix[i++]) != '\0')
    {
        if( ch == ')') p.push(ch);
        else if(isalnum(ch)) prfx[k++]=ch;
        else if( ch == '(')
        {
            while( s[top] != ')')
                prfx[k++]=p.pop();
            p.pop(); /* Remove ) */
        }
        else
        {
            /* Operator */
            while( p.pr(s[top]) >= p.pr(ch) )
                prfx[k++]=p.pop();
            p.push(ch);
        }
    }
    prfx[k]='\0';    /* Make prfx as valid string */
    strrev(prfx);
    cout<<"\n\n Prefix Exp:"<<prfx;
    getch();
}

```

Output

Infix expression a+b

Prefix Exp:+ab

## Unit 5.Queue

### 5.1 Concepts of Queue(FIFO)

5.1.1 Concepts of Queues and its basic operations.

### 5.2 Implementation of Queue

5.2.1 Simple Queue: insert, delete and display.

5.2.2 Double ended Queue: insert, delete and display.

5.2.3 Circular queue: Insert, delete and display.

---

## 5.1 Concepts of Queue(FIFO)

### 5.1.1 Concepts of Queues and its basic operations

A queue is a linear structure that follows a specific order in which operations are performed. The order is First In, First Out (FIFO). A prime example of a queue is any line of consumers waiting for a resource, where the consumer who arrived first is served first. The distinction between stacks and queues lies in the removal process. In a stack, we remove the item that was most recently added; in a queue, we remove the item that was least recently added.



## 5.2 Implementation of Queue

### 5.2.1 Simple Queue: insert, delete and display

#### OPERATIONS ON A QUEUE

The basic operations that can be performed on queue are :

1. To Insert an element in a Queue
2. To Delete an element from a Queue.
3. To Display(Traverse) all elements of a Queue.

An array  $Q[\text{max}]$  contains a maximum of MAX elements. The indices F and R indicate the positions of the front and rear elements, respectively. The variable Num represents the number of elements currently inserted in the queue. Initially, both F and R are set to 0.

1) Insert an element in a queue

Step 1 :[Check for Queue Overflow]

If  $R \geq \text{MAX}$  then

Write "Queue Overflow"

return

Step 2 :[Increment rear pointer]

$R = R + 1$

Step 3 :[Insert element]

Q[R] = NUM

Step 4 :[Set front pointer]

If F == 0 then

F=1

return

2) Delete an element from a queue

Step 1 : [Check for Queue Underflow]

If F == 0 then

Write "Queue Underflow"

return

Step 2 : [Display Deleted element]

Write "Deleted" Q[F]

Step 3 : [ Set front pointer ]

If F==R then

F = R = 0

Else

F = F + 1

### Example 5.1 Simple Queue implementation

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
#include<stdlib.h>
```

```
class queue
```

```
{
```

```
int q[10];
```

```
int r,f,max;
```

```
public:
```

```
queue()
```

```
{
```

```
cout<<"\n enter max";
```

```
cin>>max;
```

```
r = 0;
```

```
f = 0;
```

```
}
```

```
void insert(int x)
```

```
{
```

```
if(r >= max)
```

```
{
```

```
cout << "queue over flow";
```

```
return;
```

```
}
```

```
r++;
```

```

        q[r] = x;
        if(f==0)
            f=1;
    }

    void delet()
    {
        if(f==0)
        {
            cout << "queue underflow";
            return;
        }
        cout << "deleted " << q[f];

        if(f==r)
            f=r=0;
        else
            f++;
    }
    void display();
};

void queue::display()
{
    int i;
    if(f==0)
    {
        cout << " queue empty";
        return;
    }
    for(i = f; i<=r; i++)
        cout << q[i] << " ";
}

void main()
{
    clrscr();
    int ch,e;
    queue qe;
    while(1)
    {
        clrscr();
        cout << "\n 1.insert";
        cout << "\n 2.delet";
        cout << "\n 3.display";
        cout << "\n 4.exit";
        cout << "\n enter ur choice: ";
        cin >> ch;
        switch (ch)

```

```

{
case 1:
cout << "enter the element: ";
cin >> e;
qe.insert(e);
break;
case 2:
qe.delet();
break;
case 3:
qe.display();
break;
case 4:
exit(0);
default:
cout<<"\nwrong choice";
}
getch();
}
}

```

### 5.2.3 Circular queue: Insert, delete and display.

In a simple queue, memory usage can be inefficient in some situations. For instance, if we add 3 elements to a queue that has a capacity of 3, the queue becomes full. If we then remove 1 element, we cannot add new elements until the queue is completely empty, even though there is 1 available space. In contrast, a circular queue allows us to use the space left by deleted elements to insert new ones.

An array  $Q[N]$  contains a maximum of  $N$  elements. The indices  $F$  and  $R$  indicate the positions of the front and rear elements, respectively. The variable  $Num$  represents the number of elements currently inserted in the queue. Initially, both  $F$  and  $R$  are set to 0.

1) Insert an element in a queue

Step1: [Check for Queue Overflow ]

```

If (F==1 and R==N) OR (F==R+1) then
    Write "queue overflow "
    return

```

Step2 :[Reset rear pointer]

```

If R==N then
    R=1
Else
    R=R+1

```

Step 3: [Insert element]

```

Q[R]=NUM

```

Step 4: [Set front pointer]

    If F==0 then

        F=1

2) Delete an element from a queue

Step 1:[Check for Underflow]

    If F==0 then

        Write “queue underflow”

    return

Step 2 :[Display Deleted element]

    Write “Deleted ”,Q[F]

Step 3:[Reposition F and R]

    If F==R then

        F=R=0

    Else if F==N then

        F=1

    Else

        F=F+1

Step 4 : Exit

### **Example 5.2 circular queue**

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
#include<stdlib.h>
```

```
class cir
```

```
{
```

```
  int Q[10];
```

```
  int r, f,n;
```

```
  public:
```

```
    cir()
```

```
    {
```

```
        cout<<"enter value of n";
```

```
        cin>>n;
```

```
        r = 0;
```

```
        f = 0;
```

```
    }
```

```
  void insert(int num)
```

```
  {
```

```
    if((f==1 && r==n)||((f==r+1))
```

```
    {
```

```

        cout<<"\n overflow";
        return;
    }

    if(r==n)
        r=1;
    else
        r++;

    Q[r]=num;
    if(f==0)
        f=1;
}

void del()
{
    if(f==0)
    {
        cout<<"\n underflow ";
        return;
    }
    cout<<"\n Element deleted:"<<Q[f];
    if(f==r)
    {
        f=0;
        r=0;
    }
    else if(f==n)
        f=1;
    else
        f++;
}

void display();

void cir::display()
{
    int i;
    if(f==0)
    {
        cout<<"\n queue is empty ";
        return;
    }
}

```



```

    }
    else if(r>=f)
    {
        for(i=f;i<=r;i++)
            cout<<Q[i]<<" ";
    }
    else
    {
        for(i=1;i<=r;i++)
            cout<<Q[i]<<" ";
        for(i=f;i<=n;i++)
            cout<<Q[i]<<" ";
    }
}

```

```

void main()
{
    int ch,e;
    clrscr();
    cir qu;
    while(1)
    {
        clrscr();
        cout << "\n1.insert";
        cout << "\n2.delet";
        cout << "\n3.display";
        cout << "\n4.exit";
        cout << "\nenter ur choice: ";
        cin >> ch;
        switch (ch)
        {
            case 1:
                cout<<"\n Enter element :";
                cin>>e;
                qu.insert(e);
                break;
            case 2:
                qu.del();
                break;
            case 3:
                qu.display();
                break;
            case 4:

```

```

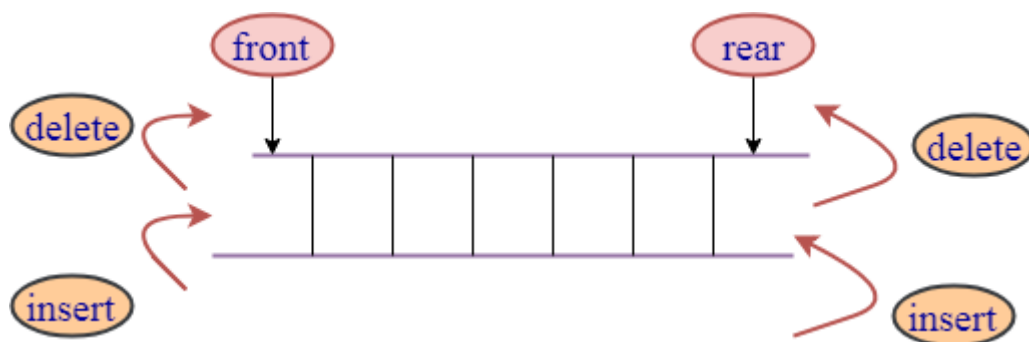
        exit(0);
        default:
            cout<<"\n wrong choice";
    }
    getch();
}
}

```

### 5.2.2 Double ended Queue: Insert, delete and display

#### Deque

Double ended queue or simply called “Deque”. The difference between Queue and Deque is that it does not follow the FIFO (First In, First Out) approach. The second feature of Deque is that we can insert and remove elements from either front or rear ends.



#### OPERATIONS ON A CIRCULAR DEQUE

The basic operations that can be performed are :

1. To Insert an element at the front in a Queue.
2. To Insert an element at the rear in a Queue.
3. To Delete an element from the front of Queue.
4. To Delete an element from the rear of Queue.

An array  $Q[N]$  contains a maximum of  $N$  elements. The indices  $F$  and  $R$  indicate the positions of the front and rear elements, respectively. The variable  $item$  represents the number of elements currently inserted in the queue. Initially, both  $F$  and  $R$  are set to  $-1$ .

- 1) To Insert an element at the front in a Queue.

Step 1 : [Check for queue overflow]

If  $((f == 0) \ \&\& \ (r == N - 1)) \ || \ (f == r + 1)$  then

Write “Queue Overflow”

return

[set  $f$  and  $r$  pointer]

Else if (f==-1) then

f=N-1

r=N-1

Else if (f==0) then

f=N-1

Else

f=f -1

Step 2 : Set Q[f] = item

Step 3 : Exit

2) To Insert an element at the rear in a Queue.

Step 1 : [Check for queue overflow]

If (((f==0) && (r==N-1)) || (f==r+1) ) then

Write "Queue Overflow"

return

[set f and r ]

Else if (f==-1) then

f=0

r=0

Else if (r==N-1) then

r=0

Else

r=r +1

Step 2 : Set Q[r] = item

Step 3 : Exit

3) To Delete an element from the front of Queue

Step 1 : [Check for queue underflow]

If f == -1 then

Write "Queue Underflow"

return

Step 2 : [Assign Deleted element]

item = Q[f]

Step 3 : [Reset f and r]

If (f ==r) then

f= r= -1

Else if (f== N-1)

f=0

Else

f=f+1

Step 4 : [Return element]

return item

Step 5: Exit

4) To Delete an element from the rear of Queue.

Step 1 : [Check for queue underflow]

    If  $f == -1$  then

        Write "Queue Underflow"

    return

Step 2 : [Assign Deleted element]

    item = Q[r]

Step 3 : [Reset f and r]

    If  $(f == r)$  then

$f = r = -1$

    Else if  $(r == 0)$

$r = N - 1$

    Else

$r = r - 1$

Step 4 : [Return element]

    return item

Step 5: Exit

### **Example 5.3 circular dequeue**

```
#include<iostream.h>
```

```
#include<stdlib.h>
```

```
#include<conio.h>
```

```
#define N 3
```

```
class deq
```

```
{
```

```
    private:
```

```
        int Q[N];
```

```
        int f,r;
```

```
    public:
```

```
        deq()
```

```
        {
```

```
             $f = -1$ ;
```

```
             $r = -1$ ;
```

```
        }
```

```
        void insert_frontend(int item);
```

```
        void insert_rearend(int item);
```

```
        int delete_frontend();
```

```
        int delete_rearend();
```

```
        void display();
```

```
};
```

```
void deq::insert_frontend(int item)
```

```
{
```

```
    if ((( $f == 0$ ) && ( $r == N - 1$ )) || ( $f == r + 1$ ))
```

```
    {
```

```
        cout<<"\nqueue overflow\n";
```

```
        return;
```

```
    }
```

```

        else if( f==-1 )/*if queue is initially empty*/
        {
            f=N-1;
            r=N-1;
        }
        else if(f==0) /* front is at first position of queue*/
            f=N-1;
        else
            f=f-1;
            Q[f]=item ;
    }/*end of insert_frontend()*/

void deq::insert_rearend(int item)
{
    if(((f==0) && (r==N-1)) || (f==r+1) )
    {
        cout<<"\nqueue overflow\n";
        return;
    }
    else if(f==-1) /*if queue is initially empty*/
    {
        f=0;
        r=0;
    }
    else if(r==N-1) /*rear is at last position of queue */
        r=0;
    else
        r=r+1;
    Q[r]=item ;
}/*end of insert_rearend()*/

int deq::delete_frontend()
{
    int item;
    if(f == -1)
    {
        cout<<"\nqueue underflow\n";
        return(0);
    }
    item=Q[f];
    if(f==r) /*queue has only one element */
    {
        f=-1;
        r=-1;
    }
    else if(f==N-1) /* front is at last position of queue*/
        f=0;
    else
        f=f+1;
}

```

```

        return item;
    } /*end of delete_frontend()*/
int deq::delete_rearend()
{
    int item;
    if(f==-1)
    {
        cout<<"\nqueue underflow\n";
        return(0);
    }
    item=Q[r];
    if(f==r) /*queue has only one element*/
    {
        f=-1;
        r=-1;
    }
    else if(r==0) /* rear is at first poition of queue*/
        r=N-1;
    else
        r=r-1;
    return item;
} /*end of delete_rearend() */

void deq::display()
{
    int i;
    if( f == -1)
    {
        cout<<"\nqueue underflow\n";
        return;
    }
    cout<<"\nqueue elements :\n";
    if(f<=r)
    {
        for(i=f;i<=r;i++)
            cout<<" "<<Q[i];
    }
    else
    {
        for(i=0;i<=r;i++)
            cout<<" "<<Q[i];
        for(i=f;i<=N-1;i++)
            cout<<" "<<Q[i];
    }
    cout<<"\n";
} /*end of display() */
/* c program to implement deque using circular array */
void main()
{

```

```

    deq q1;
    int choice,item;
    do
    {
        clrscr();
        cout<<"\n\n1.insert at the front end\n";
        cout<<"2.insert at the rear end\n";
        cout<<"3.delete from front end\n";
        cout<<"4.delete from rear end\n";
        cout<<"5.display\n";
        cout<<"6.Exit\n";
        cout<<"\nenter your choice : ";
        cin>>choice;
        switch(choice)
        {
            case 1:
                cout<<"\ninput the element for adding in queue : ";
                cin>>item;
                q1.insert_frontend(item);
                break;
            case 2:
                cout<<"\ninput the element for adding in queue : ";
                cin>>item;
                q1.insert_rearend(item);
                break;
            case 3:
                cout<<"\nelement deleted from front end is :\n"<<q1.delete_frontend();
                break;
            case 4:
                cout<<"\nelement deleted from rear end is : "<<q1.delete_rearend();
                break;
            case 5:
                q1.display();
                break;
            case 6:
                cout<<"\nHave a nice day\n";
                break;
            default:
                cout<<"\nwrong choice\n";
                break;
        }/*end of switch*/
        getch();
    }while(choice!=6);/*end of while*/
}/*end of main()*/

```

**Deque can be classified as follows:**

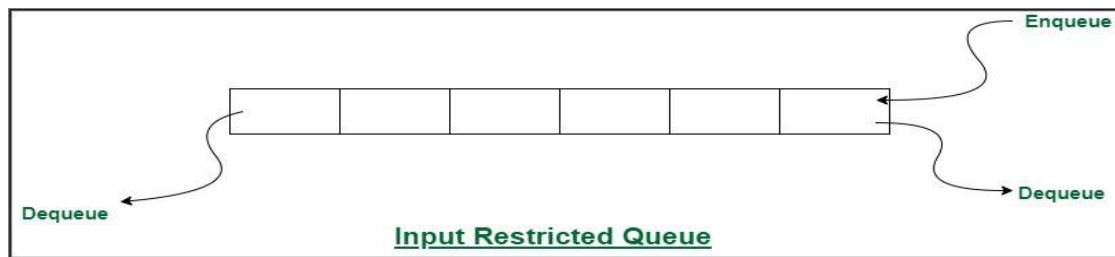
**Input-restricted Deque:** In input-restricted, deletion can be done from both the ends but insertion can be done only at one end of the queue.

The following three basic operations are performed on input restricted queue:

insertRear(): Adds an item at the rear of the queue.

deleteFront(): Deletes an item from the front of the queue.

deleteRear(): Deletes an item from the rear of the queue.



**Output-restricted Deque:** In the output-restricted queue, insertion can be done from both the ends but deletion is done only at one end i.e. the front end of the queue.

The following three basic operations are performed on output restricted queue:

insertRear(): Adds an item at the rear of the queue.

insertFront(): Adds an item at the front of the queue.

deleteFront(): Deletes an item from the front of the queue.

